

실험 전 보고서

LAB2 - 조합논리 8개

실험 주제 : LAB2 조합논리 8개 - 입력 조합 및 출력 관계 분석

실험 대상 : Counter, Clock_Divider, Register, Shift_Register, PISO,
Moore, Mealy, Segment_Scan, Integrated

작성일 : 2026년 09월 20일

검증 환경 : VS Code Vapor View, Vivado 2026.1

성명 : 권진혁

학과 : 전자전기컴퓨터공학부

학번 : 2023440007

1. Counter

· RTL의 역할

카운터는 클록 상승 에지에서 동작하며 다음과 같은 기능을 수행한다.

- rst = 1 → 카운터 값을 0으로 초기화
- enable = 0 → 현재 값 유지
- enable = 1, down = 0 → 1 증가
- enable = 1, down = 1 → 1 감소
- 4비트이므로 15 다음에는 0으로 순환
- 0에서 감소하면 15로 순환

즉, RTL은 FPGA에 구현될 실제 디지털 회로의 기능을 정의하는 역할을 한다.

· TB의 역할

- 클록 생성
- 리셋 입력
- 증가 동작 검사
- 감소 동작 검사
- 순환 동작 검사
- 유지 동작 검사
- 리셋 우선순위 검사를 수행한다.

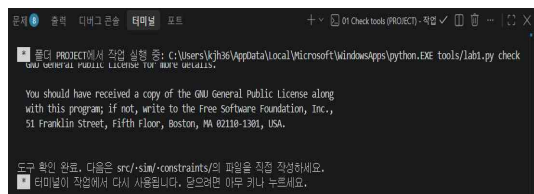
또한, 기댓값과 실제 출력값을 비교하여 오류가 발생하면 시뮬레이션을 중단하고, 모든 검사가 통과하면 PASS 로그를 출력한다.

· 초기화, 증가, 감소, 순환, 유지 예상 결과

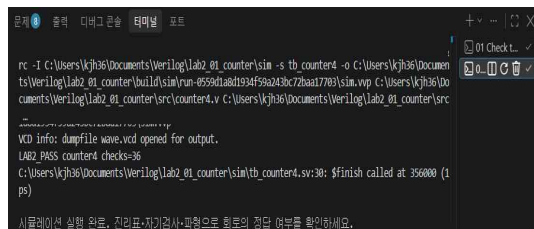
동작	입력 조건	예상 출력(value)	설명
초기화	rst=1	0	리셋 시 항상 0
증가	enable=1, down=0	이전 값+1	업 카운트
감소	enable=1, down=1	이전 값-1	다운 카운트
증가 순환	value=15 → 증가	0	4비트 오버플로
감소 순환	value=0 → 감소	15	4비트 언더플로
유지	enable=0	변화 없음	이전 값 유지

시간 구간	rst	enable	down	value
초기 상태	1	X	X	0
증가 시작	0	1	0	1
	0	1	0	2
	0	1	0	3
...	...	...	...	...
순환	0	1	0	15
다음 클록	0	1	0	0
감소 시작	0	1	1	15
다음 클록	0	1	1	14
다음 클록	0	1	1	13
유지	0	0	X	값 변화 없음
최종 리셋	1	1	X	0

· 도구 확인



· PASS Log



· 파형 분석



초기 구간에서는 rst=1 상태이므로 카운터 출력 value[3:0]가 유효한 값으로 초기화되기 전까지 X 상태를 보이다가 리셋에 의해 0으로 설정되었다. 이후 약 6ns 부근에서 rst가 0으로 비활성화되고 enable이 1로 활성화되면서 카운터 동작이 시작되었다. 이때 down=0이므로 증가 모드가 선택되어 클록의 상승 에지마다 value가 0 → 1 → 2 순

으로 증가하는 것이 확인되었다. 출력 값은 클럭 상승 에지 직후에만 변화하고 그 외 구간에서는 유지되어 순차논리회로의 특성을 정상적으로 나타냈다. 따라서 파형을 통해 리셋 동작, 카운터 활성화, 그리고 업 카운트 기능이 의도한 대로 동작함을 확인할 수 있었다.

· 설정 수정 사항 및 발생 오류

수정한 설정은 Python 경로 수정이다. 시뮬레이션 실행 중 Error Code 9009가 발생하였는데, 해당 에러의 원인은 WindowsApps의 Python 실행 별칭이 호출되었거나 실제 Python 실행 파일이 실행되지 않은 경우이다. 수정 방법은 다음과 같다.

- 1) LAB1.code-workspace 파일 열기
- 2) tasks
- 3) windows → command
- 4) Python 실행 경로를 실제 설치 경로로 수정

2. Clock Divider

· 동작 원리

clock divider는 DIVISOR=10으로 설정되어 있다. 입력 클럭이 들어올 때마다 count가 0부터 9까지 증가하며, 9 다음에는 다시 0으로 초기화된다.

- count = 0~4 : divided = 0
- count = 5~9 : divided = 1
- count = 9 : tick = 1 (1clk 폭)
- 나머지 구간 : tick = 0

예상 상태표는 다음과 같다.

count	divided	tick
0	0	0
1	0	0
2	0	0
3	0	0
4	0	0
5	1	0
6	1	0
7	1	0
8	1	0
9	1	1
다음 클럭	0	0

· 시뮬레이션 분주 주기 계산

TB에서는 입력 clk 주기가 10ns이다. 따라서 $T_{\div d} = 10ns \times 10 = 100ns$ 이다. 즉,

- 입력 클럭 주기 = 10 ns
- divided 주기 = 100 ns
- High 시간 = 50 ns
- Low 시간 = 50 ns
- Duty Cycle = 50%

tick은 count=9일 때만 1이 되므로 100ns마다 한 번 발생하며 폭은 10ns이다.

· 실제 보드 분주 주기 계산

실험 보드의 입력 clk는 1kHz이다.

$$T_{clk} = \frac{1}{1000} = 1ms \text{이다.}$$

신호	주파수	주기
입력 clk	1 kHz	1 ms
div2	500 Hz	2 ms
div10	100 Hz	10 ms
div50	20 Hz	50 ms
div1000	1 Hz	1 s
tick	1 kHz 중 1 ms 폭 펄스	1 ms

· RTL 역할

```
src > @ clock_divider.v > ...
1  `timescale 1ns/1ps
2
3  module clock_divider #(parameter integer DIVISOR = 10) (
4
5      input wire clk,
6      input wire rst,
7      output reg divided,
8      output wire tick
9
10 );
11
12     localparam integer WIDTH = $clog2(DIVISOR);
13
14     reg [WIDTH-1:0] count;
15
16     assign tick = !rst && (count == DIVISOR - 1);
17
18     always @(posedge clk) begin
19         if (rst) begin
20             count <= 0;
21             divided <= 1'b0;
22         end
23     end
24
25     else begin
26         if (count == DIVISOR - 1)
27             count <= 0;
28         else
29             count <= count + 1'b1;
30
31         if (count == DIVISOR/2 - 1)
32             divided <= 1'b1;
33         else if (count == DIVISOR - 1)
34             divided <= 1'b0;
35     end
36 endmodule
```

```
23     else begin
24         if (count == DIVISOR - 1)
25             count <= 0;
26         else
27             count <= count + 1'b1;
28
29         if (count == DIVISOR/2 - 1)
30             divided <= 1'b1;
31         else if (count == DIVISOR - 1)
32             divided <= 1'b0;
33     end
34 end
35
36 endmodule
```

```
src > @ input_frontend.v > ...
1  `timescale 1ns/1ps
2
3  module input_frontend #(parameter integer STABLE_CYCLES = 20) (
4
5      input wire clk,
6      input wire rst,
7      input wire button,
8      input wire [7:0] sw,
9      output wire reset,
10     output reg press,
11     output wire [7:0] switches
12
13 );
14
15     (* ASYNC_REG = "TRUE" *) reg [1:0] reset_pipe;
16     (* ASYNC_REG = "TRUE" *) reg button_meta, button_sync;
17     (* ASYNC_REG = "TRUE" *) reg [7:0] sw_meta, sw_sync;
18
19     localparam integer WIDTH =
20         (STABLE_CYCLES < 2) ? 1 : $clog2(STABLE_CYCLES);
21
22     reg [WIDTH-1:0] count;
```

```
23     reg accepted;
24
25     assign reset = reset_pipe[1];
26     assign switches = sw_sync;
27
28     // Asynchronous assertion; release follows two main-clock edges.
29     always @(posedge clk or posedge rst) begin
30         if (rst)
31             reset_pipe <= 2'b11;
32         else
33             reset_pipe <= {reset_pipe[0], 1'b0};
34     end
35
36     always @(posedge clk or posedge reset) begin
37         if (reset) begin
38             button_meta <= 0;
39             button_sync <= 0;
40             sw_meta <= 0;
41             sw_sync <= 0;
42         end
43     end
44
45     else begin
46         button_meta <= button;
47         button_sync <= button_meta;
48         sw_meta <= sw;
49         sw_sync <= sw_meta;
50     end
51
52     always @(posedge clk or posedge reset) begin
53         if (reset) begin
54             count <= 0;
55             accepted <= 0;
56             press <= 0;
57         end
58     end
59
60     else begin
61         count <= count + 1'b1;
62         accepted <= button_sync;
63         count <= 0;
64         press <= button_sync;
65     end
66
67     else begin
68         count <= count + 1'b1;
69     end
70
71 end
72
73 endmodule
```

```
44         button_meta <= button;
45         button_sync <= button_meta;
46         sw_meta <= sw;
47         sw_sync <= sw_meta;
48     end
49
50     always @(posedge clk or posedge reset) begin
51         if (reset) begin
52             count <= 0;
53             accepted <= 0;
54             press <= 0;
55         end
56     end
57
58     else begin
59         press <= 0;
60     end
61
62     else begin
63         count <= count + 1'b1;
64         accepted <= button_sync;
65         count <= 0;
66         press <= button_sync;
67     end
68
69     else begin
70         count <= count + 1'b1;
71     end
72
73 endmodule
```

```
60         if (button_sync == accepted)
61             count <= 0;
62         else if (count == STABLE_CYCLES - 1) begin
63             accepted <= button_sync;
64             count <= 0;
65             press <= button_sync;
66         end
67     end
68
69     else begin
70         count <= count + 1'b1;
71     end
72
73 end
74
75 endmodule
```

```

1 `timescale 1ns/1ps
2
3 module lab2_clock_divider #(parameter integer STABLE_CYCLES = 20,
4                             DIVISOR = 1000) (
5
6     input wire clk,
7     input wire rst,
8     input wire button,
9     input wire [7:0] sw,
10    output wire [7:0] led
11
12 );
13
14 wire reset, press;
15 wire [7:0] switches;
16
17 input_frontend #(
18     .STABLE_CYCLES(STABLE_CYCLES)
19 ) inputs (
20     clk,
21     rst,
22     button,

```

```

59     .tick()
60 );
61
62 clock_divider #(
63     .DIVISOR(DIVISOR)
64 ) core (
65     .clk(clk),
66     .rst(reset),
67     .divided(divided),
68     .tick(tick)
69 );
70
71 assign led = {3'b000, tick, divided, div50, div10, div2};
72
73 endmodule

```

count 증가, divided 생성, tick 생성, 버튼 및 리셋 처리 및 LED 출력 연결의 역할을 한다.

```

23     sw,
24     reset,
25     press,
26     switches
27 );
28
29 wire divided;
30 wire tick;
31 wire div2;
32 wire div10;
33 wire div50;
34
35 clock_divider #(
36     .DIVISOR(2)
37 ) ratio2 (
38     .clk(clk),
39     .rst(reset),
40     .divided(div2),

```

· TB의 역할

- 주요 검증 항목
- DIVISOR=10 동작 확인
- DIVISOR=2 동작 확인
- 듀티비 검사
- tick 발생 검사
- 리셋 동작 검사
- 총 129개 비교 수행

· Simulation.json

컴파일할 RTL 지용, 사용할 TB 지정한 후 최상위 모듈 지정한다.

```

41     .tick()
42 );
43
44 clock_divider #(
45     .DIVISOR(10)
46 ) ratio10 (
47     .clk(clk),
48     .rst(reset),
49     .divided(div10),
50     .tick()
51 );
52
53 clock_divider #(
54     .DIVISOR(50)
55 ) ratio50 (
56     .clk(clk),
57     .rst(reset),
58     .divided(div50),

```

· XDC

XDC는 FPGA 핀과 타이밍 제약을 정의하는 파일이다.

- 역할
- clk → B6 핀 연결
- rst → K4 핀 연결
- button → N8 핀 연결
- LED 핀 연결
- 1 kHz 클록 제약 설정

· 정상, 변경, 원복 로그 비교

구분	수정 내용	결과
정상	DIVISOR/2-1 사용	LAB2_PASS checks=129
변경	DIVISOR/2 사용	duty 검사 실패
원복	DIVISOR/2-1 복구	LAB2_PASS checks=129

정상

```
1 VCD info: dumpfile wave.vcd opened for output.
2 LAB2_PASS clock_divider checks=129
```

변경

```
1 LAB2_FAIL
2 minimum divisor duty
```

원복

```
1 VCD info: dumpfile wave.vcd opened for output.
2 LAB2_PASS clock_divider checks=129
```

· 전체 파형 및 듀티, tick, 리셋 확대 해석

- 전체 파형

관찰 신호

clk

rst

divided

tick

div2

tick2

확인 사항

divided 주기 100ns

tick 주기 100ns

tick 폭 10ns

리셋 후 정상 재시작

- 듀티 사이클 해석

55ns 상승

105ns 하강

155ns 상승

한다.

따라서

High = 50ns

Low = 50ns

DutyCycle = 50% Duty로 계산된다.

- tick 해석

tick 발생 구간은

95~105ns

195~205ns

295~305ns이다.

특징은

1클록 폭

count=9에서만 발생

Enable 신호 역할 수행이다.

- 리셋 해석

376ns : rst=1

385ns : count 초기화

divided 초기화

tick 강제 0

435ns : divided 재동작한다.

따라서 리셋이 정상적으로 회로 상태를 초기화하고 있음을 확인할 수 있다.

· 1kHz 보드 설정, LED 예상 동작 및 측정 방법

- 보드 설정

입력 클록

1 1 kHz

클록 주기

1 1 ms

- LED 예상 동작

LED	신호	예상 동작
LED0	div2	500Hz 점멸
LED1	div10	100Hz 점멸
LED2	div50	20Hz 점멸
LED3	div1000	1Hz 점멸
LED4	tick	1ms 폭 펄스
LED5	0	항상 OFF
LED6	0	항상 OFF
LED7	0	항상 OFF

- 측정 방법

LED0 (500Hz)

사람 눈으로 확인 어려움

오실로스코프 사용

LED1 (100Hz)

오실로스코프 권장

LED2 (20Hz)

LED 깜빡임 관찰 가능

LED3 (1Hz)

0.5초 ON

0.5초 OFF

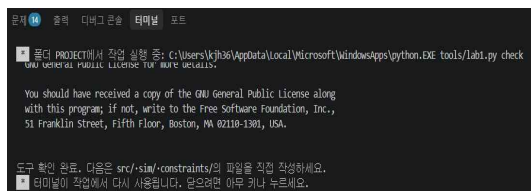
육안으로 쉽게 확인 가능

LED4 (tick)

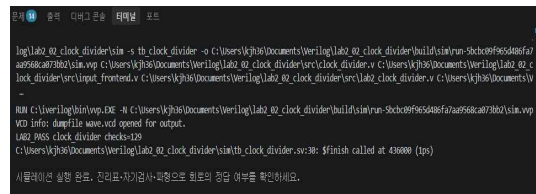
폭이 1ms라 육안 확인 불가

오실로스코프 사용

· 도구 확인



· 시뮬레이션 결과



· 파형



초기 구간 파형을 확인한 결과 clk는 10 ns 주기로 정상 동작하였으며, rst는 시뮬레이션 시작 직후 활성화되었다가 해제되었다. divided와 tick은 아직 10분주 카운트가 완료되지 않은 구간이므로 출력 변화가 발생하지 않았으며, 이는 예상 결과와 일치하였다. 반면 div2와 tick2는 2분주 회로의 동작을 보여 주며 약 20ns 주기의 50% Duty Cycle 파형이 관찰되었다. 따라서 리셋 회로와 최소 분주 회로가 정상 동작함을 확인하였다.

3. Register

· 레지스터 동작 원리

4비트 레지스터 2개(stored, value)를 사용한다.

- stored : 입력 데이터를 저장
- value : stored에 저장된 값을 전달받아 저장
- 모든 동작은 clk 상승 에지에서 수행
- rst가 가장 높은 우선순위를 가짐
- load는 data_in을 stored에 저장

- transfer는 stored 값을 value에 전달

· 상태표

rst	load	transfer	다음 stored	다음 value
1	X	X	0	0
0	0	0	유지	유지
0	1	0	data_in	유지
0	0	1	유지	이전 stored
0	1	1	data_in	이전 stored

· 동시 동작(load = 1, transfer = 1) 계산 과정

- 초기 상태

stored = A, value = 0, data_in = 3

- 제어 신호

load = 1, transfer = 1

- 상승 에지 발생 전

stored = A, value = 0

- At RTL

stored <= data_in;

2value <= stored;

가 동시에 실행된다.

nonblocking assignment(<=)는 에지 직전 값을 읽는다. 따라서 'stored <= 3, value <= A'가 예약된다. 에지 이후 결과 'stored = 3, value = A'가 된다. 다음 클록에서도 load = 1, transfer = 1이면 'stored = 3, value = 3'이 된다. 즉, value는 항상 이전 stored 값을 받는다.

· RTL

```

1 `timescale 1ns/1ps
2 module register_pair (
3
4     input wire clk, input wire rst, input wire load, input wire transfer, input wire [3:0] data_in,
5     output reg [3:0] stored, output reg [3:0] value
6 );
7
8 always @(posedge clk) begin
9     if (rst) begin
10         stored <= 4'd0;
11         value <= 4'd0;
12     end
13     else begin
14         if (load)
15             stored <= data_in;
16         if (transfer)
17             value <= stored;
18     end
19 end
20 endmodule

```

실제 레지스터 동작을 구현한 핵심 모듈이다.

- rst → stored와 value 초기화

- load → data_in 저장

- transfer → stored 값을 value로 전달
실제 회로(Flip-Flop)의 동작을 기술한다.

```

1 `timescale 1ns/1ps
2
3 module input_frontend #(parameter integer STABLE_CYCLES = 20) (
4
5     input wire clk,
6     input wire rst,
7     input wire button,
8     input wire [7:0] sw,
9     output wire reset,
10    output reg press,
11    output wire [7:0] switches
12 );
13
14 (* ASYNC_REG = "TRUE" *) reg [1:0] reset_pipe;
15 (* ASYNC_REG = "TRUE" *) reg button_meta, button_sync;
16 (* ASYNC_REG = "TRUE" *) reg [7:0] sw_meta, sw_sync;
17
18 localparam integer WIDTH =
19     (STABLE_CYCLES < 2) ? 1 : $clog2(STABLE_CYCLES);
20
21 reg [WIDTH-1:0] count;
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43

```

```

23 reg accepted;
24
25 assign reset = reset_pipe[1];
26 assign switches = sw_sync;
27
28 // Asynchronous assertion; release follows two main-clock edges.
29 always @(posedge clk or posedge rst) begin
30     if (rst)
31         reset_pipe <= 2'b11;
32     else
33         reset_pipe <= {reset_pipe[0], 1'b0};
34 end
35
36 always @(posedge clk or posedge reset) begin
37     if (reset) begin
38         button_meta <= 0;
39         button_sync <= 0;
40         sw_meta <= 0;
41         sw_sync <= 0;
42     end
43     else begin

```

```

44         button_meta <= button;
45         button_sync <= button_meta;
46         sw_meta <= sw;
47         sw_sync <= sw_meta;
48     end
49
50
51     always @(posedge clk or posedge reset) begin
52         if (reset) begin
53             count <= 0;
54             accepted <= 0;
55             press <= 0;
56         end
57         else begin
58             press <= 0;
59
60             if (button_sync == accepted)
61                 count <= 0;
62
63             else if (count == STABLE_CYCLES - 1) begin
64                 accepted <= button_sync;
65                 count <= 0;
66                 press <= button_sync;
67             end
68             else
69                 count <= count + 1'b1;
70         end
71     end
72 endmodule

```

보드의 버튼과 스위치를 FPGA 내부 클럭에 동기화한다.

- 버튼 디바운싱
- 동기화
- press 신호 생성

버튼을 한 번 눌렀을 때 한 번만 동작하도록 만든다.

```

1 `timescale 1ns/1ps
2
3 module lab2_register #(parameter integer STABLE_CYCLES = 20) (
4     input wire clk, input wire rst, input wire button, input wire [7:0] sw,
5     output wire [7:0] led
6 );
7
8
9
10 wire reset; wire press; wire [7:0] switches;
11
12 input_frontend #(
13     .STABLE_CYCLES(STABLE_CYCLES)
14 ) inputs (
15     clk,
16     rst,
17     button,
18     sw,
19     reset,
20     press,
21     switches
22 );

```

```

23
24 wire [3:0] stored;
25 wire [3:0] value;
26
27 register_pair core(
28     clk,
29     reset,
30     press && switches[0],
31     press && switches[1],
32     switches[7:4],
33     stored,
34     value
35 );
36
37 assign led = {stored, value};
38
39 endmodule

```

최상위(top) 모듈이다.

- input_frontend 연결
- register_pair 연결
- LED 출력 연결

실제 FPGA에 다운로드되는 회로이다

· TB

```

1 `timescale 1ns/1ps
2
3 module tb_register_pair;
4
5     reg clk=0, rst=1;
6     integer checks=0;
7
8     always #5 clk=~clk;
9
10    task step;
11    begin
12        @(posedge clk);
13        #1;
14    end
15    endtask
16
17    task check(input condition, input [8*100-1:0] description);
18    begin
19        checks=checks+1;
20        if (condition != 1'b1) begin
21            $display("LAB2_FAIL %0s time=%0t",description,$time);
22            $fatal(1,"check failed");
23        end
24    end
25 endmodule

```

```

23     end
24 endtask
25
26 task finish;
27 begin
28     $display("LAB2_PASS register_pair checks=%0d",checks);
29     $finish;
30 end
31 endtask
32
33 initial begin
34     $dumpfile("wave.vcd");
35     $dumpvars(0,tb_register_pair);
36 end
37
38 initial begin
39     #100000;
40

```

```

41         $fatal(1,"watchdog timeout");
42     end
43
44     reg load=0, transfer=0;
45     reg [3:0] data_in=0;
46     wire [3:0] stored, value;
47
48     register_pair dut(
49         clk,
50         rst,
51         load,
52         transfer,
53         data_in,
54         stored,
55         value
56     );
57
58     initial begin
59         step;

```

```

60         check(condition: {stored,value}==8'h00,description: "reset");
61
62         rst=0;
63
64         data_in=4'h4;
65         load=1;
66         step;
67         check(condition: {stored,value}==8'h00,description: "load does not transfer");
68
69         load=0;
70         data_in=4'h3;
71         transfer=1;
72         step;
73         check(condition: {stored,value}==8'h00,description: "transfer stored not live input");
74
75         load=1;
76         step;
77         check(condition: {stored,value}==8'h3a,description: "simultaneous uses old stored");

```

```

78         step;
79         check(condition: {stored,value}==8'h33,description: "next edge transfers new stored");
80
81         load=0;
82         transfer=0;
83         data_in=4'hf;
84         step;
85         check(condition: {stored,value}==8'h33,description: "both hold");
86
87         rst=1;
88         load=1;
89         transfer=1;
90         step;
91         check(condition: {stored,value}==8'h00,description: "reset beats both controls");
92
93         finish;
94     end
95
96
97 endmodule

```

시뮬레이션 전용 코드이다.

- 클럭 생성
- 입력 벡터 제공
- 기대값 자동 비교
- PASS/FAIL 판정

총 7개의 검사를 수행하며 모두 통과하면 'LAB2_PASS register_pair checks=7'이 출력된다.

· json

```

1 {
2   "sources": ["src/register_pair.v", "src/input_frontend.v", "src/lab2_register.v"],
3   "testbench": "sim/tb_register_pair.sv",
4   "simulation_top": "tb_register_pair"
5 }
6

```

시뮬레이터에게 어떤 파일을 사용할지 알려주는 설정 파일이다.

- sources : RTL 파일 목록
 - testbench : TB 파일
 - simulation_top : 최상위 TB 모듈 이름
- Icarus Verilog가 컴파일할 파일을 지정한다.

· XDC

```

constraints > lab2_register.xdc
1  set_property PACKAGE_PIN B6 [get_ports {clk}]
2  set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
3
4  set_property PACKAGE_PIN K4 [get_ports {rst}]
5  set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
6
7  set_property PACKAGE_PIN N8 [get_ports {button}]
8  set_property IOSTANDARD LVCMOS33 [get_ports {button}]
9
10 set_property PACKAGE_PIN U4 [get_ports {sw[0]}]
11 set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
12
13 set_property PACKAGE_PIN V4 [get_ports {sw[1]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
15
16 set_property PACKAGE_PIN W1 [get_ports {sw[2]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
18
19 set_property PACKAGE_PIN W4 [get_ports {sw[3]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
21
22 set_property PACKAGE_PIN T1 [get_ports {sw[4]}]

```

```

23 set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
24
25 set_property PACKAGE_PIN U2 [get_ports {sw[5]}]
26 set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
27
28 set_property PACKAGE_PIN W3 [get_ports {sw[6]}]
29 set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
30
31 set_property PACKAGE_PIN Y1 [get_ports {sw[7]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
33
34 set_property PACKAGE_PIN N5 [get_ports {led[0]}]
35 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
36
37 set_property PACKAGE_PIN M1 [get_ports {led[1]}]
38 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
39
40 set_property PACKAGE_PIN M3 [get_ports {led[2]}]
41 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
42

```

```

43 set_property PACKAGE_PIN M7 [get_ports {led[3]}]
44 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
45
46 set_property PACKAGE_PIN N7 [get_ports {led[4]}]
47 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
48
49 set_property PACKAGE_PIN M2 [get_ports {led[5]}]
50 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
51
52 set_property PACKAGE_PIN M4 [get_ports {led[6]}]
53 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
54
55 set_property PACKAGE_PIN L4 [get_ports {led[7]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
57
58 create_clock -name trainer_1khz -period 1000000.000 [get_ports clk]
59
60 set_false_path -from [get_ports {rst button sw[*]}]

```

FPGA 핀 배치를 정의한다.

ex)

- clk → B6
- rst → K4
- button → N8
- switch → DIP SW
- led → LED 출력

RTL 포트와 실제 FPGA 핀을 연결한다.

· 정상, 변경, 원복 실행 로그 비교

- 정상 실행

```

C:\Users\kjh36\appdata\local\Microsoft\WindowsApps\python.exe tools\lbt.py simulate
Run C:\Verilog\bin\vvp.EXE -N C:\Users\kjh36\Documents\Verilog\lab2_03_register\build\sim-run-10aff1e4809c485cb02245202cc74\sim.vp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS register_pair checks=7
C:\Users\kjh36\Documents\Verilog\lab2_03_register\sim\lb_register_pair_sv:30: $finish called at 6000 (tps)

시뮬레이션 실행 완료. 전리표-기타검사-파형으로 회로의 동작 여부를 확인하세요.
Wave C:\Users\kjh36\Documents\Verilog\lab2_03_register\build\sim\wave.vcd
타이밍이 정합되지 다시 시뮬레이션, 문으러면 아무 키나 누르세요.

```

- 변경 실행

value <= stored;

..

를

value <= data_in;

로 변경한다.

```

lab2_03_register\sim -s tb register_pair -o C:\Users\kjh36\Documents\Verilog\lab2_03_register\build\sim-run-10aff1e4809c485cb02245202cc74\sim.vp
sim.vp C:\Users\kjh36\Documents\Verilog\lab2_03_register\src\register_pair.v C:\Users\kjh36\Documents\Verilog\lab2_03_register\src\input_fr
ontend.v C:\Users\kjh36\Documents\Verilog\lab2_03_register\src\lab2_register.v C:\Users\kjh36\Documents\Verilog\lab2_03_register\sim\lb_regi
ster_pair_sv
Run C:\Verilog\bin\vvp.EXE -N C:\Users\kjh36\Documents\Verilog\lab2_03_register\build\sim-run-10aff1e4809c485cb02245202cc74\sim.vp
VCD info: dumpfile wave.vcd opened for output.
LAB2_FAIL transfer stored not live input time=26000
FATAL: C:\Users\kjh36\Documents\Verilog\lab2_03_register\sim\lb_register_pair_sv:22: check failed
Time: 26000 Scope: tb_register_pair.check

```

- 원복 실행

```

Run C:\Verilog\bin\vvp.EXE -N C:\Users\kjh36\Documents\Verilog\lab2_03_register\src-1 C:\Users\kjh36\Documents\Verilog\lab2_03_register\sim -s tb register_pair -o C:\Users\kjh36\Documents\Verilog\lab2_03_register\build\sim-run-d3a6d80b9f46d9600014979836cc\sim.vp
sim.vp C:\Users\kjh36\Documents\Verilog\lab2_03_register\src\register_pair.v C:\Users\kjh36\Documents\Verilog\lab2_03_register\src\input_fr
ontend.v C:\Users\kjh36\Documents\Verilog\lab2_03_register\src\lab2_register.v C:\Users\kjh36\Documents\Verilog\lab2_03_register\sim\lb_regi
ster_pair_sv
Run C:\Verilog\bin\vvp.EXE -N C:\Users\kjh36\Documents\Verilog\lab2_03_register\build\sim-run-d3a6d80b9f46d9600014979836cc\sim.vp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS register_pair checks=7
C:\Users\kjh36\Documents\Verilog\lab2_03_register\sim\lb_register_pair_sv:30: $finish called at 6000 (tps)

```

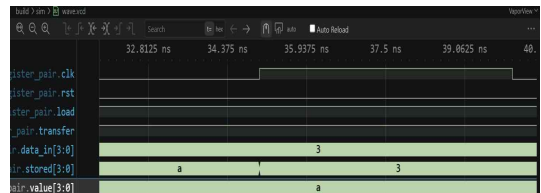
· 전체 파형 및 확대 파형 해석

- 25ns



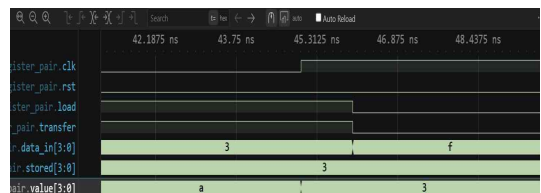
입력은 이미 3으로 바뀌었지만, transfer는 저장된 stored 값을 전달한다. 따라서 'value = A'가 된다. 입력을 직접 전달하는 회로가 아님을 확인할 수 있다.

- 35ns



제어 부문에서 'load = 1', 'transfer = 1' 상승 에지 발생. 에지 직후 'stored = 3', 'value = A' 동시에 동작하지만, value는 이전 stored 값을 읽기 때문에 A가 유지된다.

- 45ns



다시 'load = 1', 'transfer = 1' 실행 에지 직후 'stored = 3', 'value = 3' 이전 에지에서 stored가 3으로 바뀌었기 때문에 이번

에는 value도 3이 된다.

· 보드 입력 순서 및 예상 LED 계산

스위치(sw[7:0])	의미	버튼 실행 후 LED[7:0]
A1	data=A, load=1	A0
32	data=3, transfer=1	AA
33	data=3, load=1, transfer=1	3A
33 (한 번 더)	동일 동작 반복	33
F0	load=0, transfer=0	33 유지
K4 Reset	rst=1	00

- A1 → stored=A 저장 → LED=A0
- 32 → stored가 value 전달 → LED=AA
- 33 → stored=3, value=A → LED=3A
- 33 재실행 → value=3 → LED=33
- F0 → 제어신호 없음 → 유지 → LED=33
- Reset → stored=value=0 → LED=00

4. Shift Register

- enable·rst 별 다음 상태표와 8 → 4 → A → 5 계산

- 다음 상태표

rst	enable	다음 value[3:0]
1	X	0000
0	0	현재 값 유지
0	1	{serial_in, 현재 value[3:1]}

4비트 시프트 레지스터는 클록 상승 에지에서 동작한다. rst가 1이면 항상 0으로 초기화되며, rst=0 상태에서 enable=1일 때만 새로운 직렬 입력(serial_in)을 받아 저장 값을 갱신한다.

- 8 → 4 → A → 5 계산 과정

① 초기화

value = 0000

② serial_in=1, enable=1

{1,000} = 1000

따라서

value = 1000 = 0x8

③ serial_in=0, enable=1

이전 값

1000

에 대해

{0,100} = 0100

따라서

value = 0100 = 0x4

④ enable=0

serial_in이 1로 변해도 저장값 유지

value = 0100 = 0x4

⑤ serial_in=1, enable=1

이전 값 0100에 대해

{1,010} = 1010

따라서

value = 1010 = 0xA

⑥ serial_in=0, enable=1

이전 값 1010에 대해

{0,101} = 0101

따라서

value = 0101 = 0x5

· RTL

```
src > shift_register4.v 2.U X lab2_shift_register.v 3.U X simulation.json M X wave.vcd
1 `timescale 1ns/1ps
2
3 module shift_register4 (
4
5     input wire clk,
6     input wire rst,
7     input wire enable,
8     input wire serial_in,
9     output reg [3:0] value
10
11 );
12
13 always @(posedge clk) begin
14     if (rst)
15         value <= 4'd0;
16     else if (enable)
17         value <= {serial_in, value[3:1]};
18 end
19
20 endmodule
```

- 4비트 시프트 레지스터 구현
- rst=1 → value=0000
- enable=1 → value <= {serial_in, value[3:1]}
- enable=0 → 현재 값 유지

```
src > input_frontend.v 2.U X lab2_shift_register.v 1.U X simulation.json M X wave.vcd X shift_register4.v 1.U X
1 `timescale 1ns/1ps
2
3 module input_frontend #(parameter integer STABLE_CYCLES = 20) (
4
5     input wire clk, input wire rst, input wire button, input wire [7:0] sw,
6     output wire reset, output reg press, output wire [7:0] switches
7
8 );
9
10 (* ASYNC_REG = "TRUE" *) reg [1:0] reset_pipe;
11 (* ASYNC_REG = "TRUE" *) reg button_meta, button_sync;
12 (* ASYNC_REG = "TRUE" *) reg [7:0] sw_meta, sw_sync;
13
14 localparam integer WIDTH =
15     (STABLE_CYCLES < 2) ? 1 : $clog2(STABLE_CYCLES);
16
17 reg [WIDTH-1:0] count;
18 reg accepted;
19
20 assign reset = reset_pipe[1];
21 assign switches = sw_sync;
22
```

```
23 always @(posedge clk or posedge rst) begin
24     if (rst)
25         reset_pipe <= 2'b11;
26     else
27         reset_pipe <= {reset_pipe[0], 1'b0};
28 end
29
30 always @(posedge clk or posedge reset) begin
31     if (reset) begin
32         button_meta <= 0;
33         button_sync <= 0;
34         sw_meta <= 0;
35         sw_sync <= 0;
36     end
37     else begin
38         button_meta <= button;
39         button_sync <= button_meta;
40         sw_meta <= sw;
41         sw_sync <= sw_meta;
42     end
43 end
```

```
44 always @(posedge clk or posedge reset) begin
45     if (reset) begin
46         count <= 0;
47         accepted <= 0;
48         press <= 0;
49     end
50     else begin
51         press <= 0;
52
53         if (button_sync == accepted)
54             count <= 0;
55         else if (count == STABLE_CYCLES - 1) begin
56             accepted <= button_sync;
57             count <= 0;
58             press <= button_sync;
59         end
60         else
61             count <= count + 1'b1;
62     end
63 end
64 endmodule
```

- 버튼 디바운싱
- 비동기 입력 동기화
- 버튼 1회 입력을 enable 펄스로 변환

```
src > input_frontend.v 2.U X lab2_shift_register.v 1.U X simulation.json M X wave.vcd X shift_register4.v 1.U X
1 `timescale 1ns/1ps
2 module lab2_shift_register #(parameter integer STABLE_CYCLES = 20) (
3
4     input wire clk, input wire rst, input wire button, input wire [7:0] sw,
5     output wire [7:0] led
6
7 );
8 wire reset; wire press; wire [7:0] switches;
9
10 input_frontend #(
11     .STABLE_CYCLES(STABLE_CYCLES)
12 ) inputs (
13     clk, rst, button, sw, reset, press, switches
14 );
15 wire [3:0] value;
16 shift_register4 core(
17     clk, reset, press, switches[7], value
18 );
19 assign led = {4'b0000, value};
20 endmodule
```

- 최상위 모듈
- input_frontend와 shift_register4 연결
- 보드 입출력 포트 관리

· TB

```
sim > input_frontend.v 2.U X lab2_shift_register.v 1.U X simulation.json M X wave.vcd X shift_register4.v 1.U X
1 `timescale 1ns/1ps
2
3 module tb_shift_register4;
4
5     reg clk=0, rst=1;
6     integer checks=0;
7
8     always #5 clk<-clk;
9
10    task step;
11    begin
12        @(posedge clk);
13        #1;
14    end
15    endtask
16
17    task check(input condition, input [8*100-1:0] description);
18    begin
19        checks=checks+1;
20        if (condition != 1'b1) begin
21            $display("LAB2_FAIL %0s time=%0t",description,$time);
22            $fatal(1,"check failed");
23        end
24    end
25 endmodule
```

```

23     end
24 end
25 endtask
26
27 task finish;
28 begin
29     $display("LAB2_PASS shift_register4 checks=%0d",checks);
30     $finish;
31 end
32 endtask
33
34 initial begin
35     $dumpfile("wave.vcd");
36     $dumpvars(0,tb_shift_register4);
37 end
38
39 initial begin
40     #100000;

```

```

41     $fatal(1,"watchdog timeout");
42 end
43
44 reg enable=0, serial_in=0;
45 wire [3:0] value;
46
47 shift_register4 dut(
48     clk,
49     rst,
50     enable,
51     serial_in,
52     value
53 );
54
55 initial begin
56     step;
57     check(condition: value==0,description: "reset");
58
59     rst=0;

```

```

60     enable=1;
61
62     serial_in=1;
63     step;
64     check(condition: value==4'b1000,description: "input enters MSB");
65
66     serial_in=0;
67     step;
68     check(condition: value==4'b0100,description: "move toward LSB");
69
70     enable=0;
71     serial_in=1;
72     step;
73     check(condition: value==4'b0100,description: "hold");
74
75     enable=1;
76     step;
77     check(condition: value==4'b1010,description: "retain old stage values");
78

```

```

79     serial_in=0;
80     step;
81     check(condition: value===4'b0101,description: "four bit history");
82
83     repeat(4) step;
84     check(condition: value===0,description: "flush with four zeros");
85
86     serial_in=1;
87     rst=1;
88     step;
89     check(condition: value===0,description: "reset priority");
90
91     finish;
92 end
93
94 endmodule

```

- 클럭 생성
- 리셋 생성
- serial_in 입력 제공

- 기대값과 실제값 비교
- 오류 발생 시 LAB2_FAIL 출력
- 모든 검사를 통과하면 LAB2_PASS 출력

· json

```

1 {
2   "sources": ["src/shift_register4.v", "src/input_frontend.v", "src/lab2_shift_register.v"],
3   "testbench": "sim/tb_shift_register4.sv",
4   "simulation_top": "tb_shift_register4"
5 }
6

```

- 컴파일할 RTL 등록
- TB 지정
- 최상위 시뮬레이션 모듈 지정

· XDC

```

1 set_property PACKAGE_PIN B6 [get_ports {clk}]
2 set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
3
4 set_property PACKAGE_PIN K4 [get_ports {rst}]
5 set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
6
7 set_property PACKAGE_PIN N8 [get_ports {button}]
8 set_property IOSTANDARD LVCMOS33 [get_ports {button}]
9
10 set_property PACKAGE_PIN U4 [get_ports {sw[0]}]
11 set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
12
13 set_property PACKAGE_PIN V4 [get_ports {sw[1]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
15
16 set_property PACKAGE_PIN W1 [get_ports {sw[2]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
18
19 set_property PACKAGE_PIN W4 [get_ports {sw[3]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
21
22 set_property PACKAGE_PIN T1 [get_ports {sw[4]}]

```

```

23 set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
24
25 set_property PACKAGE_PIN U2 [get_ports {sw[5]}]
26 set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
27
28 set_property PACKAGE_PIN W3 [get_ports {sw[6]}]
29 set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
30
31 set_property PACKAGE_PIN Y1 [get_ports {sw[7]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
33
34 set_property PACKAGE_PIN N5 [get_ports {led[0]}]
35 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
36
37 set_property PACKAGE_PIN M1 [get_ports {led[1]}]
38 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
39
40 set_property PACKAGE_PIN M3 [get_ports {led[2]}]
41 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
42
43 set_property PACKAGE_PIN M7 [get_ports {led[3]}]

```

```

44 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
45
46 set_property PACKAGE_PIN N7 [get_ports {led[4]}]
47 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
48
49 set_property PACKAGE_PIN M2 [get_ports {led[5]}]
50 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
51
52 set_property PACKAGE_PIN M4 [get_ports {led[6]}]
53 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
54
55 set_property PACKAGE_PIN L4 [get_ports {led[7]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
57
58 create_clock -name trainer_1khz -period 1000000.000 [get_ports clk]
59
60 set_false_path -from [get_ports {rst button sw[*]}]

```

- FPGA 핀 지정
- LED 연결
- 스위치 연결
- 버튼 연결
- 클록 주기 지정

· 정상, 변경, 원복 비교

- 정상 실행

[illegible]

- ## - 변경 실행

‘{serial_in,value[3:1]}’를
‘{value[2:0],serial_in}’로 수정한다.

[illegible]

- 원복

[illegible]

· 파형 해석



- 전체 파형

시간	value
6 ns	0
16 ns	8
26 ns	4
36 ns	4
46 ns	A
56 ns	5
96 ns	0
106 ns	0

- Enable 유지 구간

35ns 상승 에지에서 ‘enable = 0’, ‘serial_in = 1’이어도 ‘value = 0100’ 상태를 유지한다. 이는 enable이 0이면 레지스터가 값을 저장만 하고 갱신하지 않음을 보여준다.

- Reset 우선 구간

105 ns 상승 에지에서 ‘rst = 1’, ‘enable = 1’, ‘serial_in = 1’이지만, ‘value = 0000’이 된다. 즉, enable보다 rst가 우선 적용됨을 확인할 수 있다.

· 보드 조작 및 예상 LED 결과

- 핀 매핑

기능	보드 핀
Serial Input	SW1
Shift Button	N8
Reset	K4

- ### - 실험 순서 및 예상 결과

동작	LED[7:0]
K4 눌러 초기화	00
SW1=1, N8 1회	08
SW1=0, N8 1회	04
SW1=1, N8 1회	0A
SW1=0, N8 1회	05
SW1=0, N8 4회	02 ~ 01 ~ 00 ~ 00

N8을 누르지 않고 SW1만 변경하면 LED 값은 변하지 않아야 한다. 이는 enable 신호가 발생하지 않아 레지스터가 현재 값을 유지하기 때문이다.

5. PISO

· 회로의 동작

PISO는 4비트 병렬 데이터를 한 번에 저장한 후, 클록이 들어올 때마다 MSB부터 1비트씩 직렬 출력하는 회로이다. load=1이면 병렬 입력을 저장하고, enable=1이면 왼쪽으로 shift하며 빈 LSB는 0으로 채운다. serial_out은 항상 저장값(value)의 MSB(value[3])를 출력한다.

· 제어 우선순위 표

rst	load	enable	다음 상태(value)
1	X	X	0000
0	1	X	data_in
0	0	1	{value[2:0],0}
0	0	0	유지

load와 enable이 동시에 1이면 load가 우선한다.

· 상태 전이 예측

- Load 후 상태
data_in = 1010
value = 1010
serial_out = 1

- Shift 과정

단계	현재 상태(value)	serial_out
Load 직후	1010	1
1회 Shift	0100	0
2회 Shift	1000	1
3회 Shift	0000	0
4회 Shift	0000	0

1010의 비트열은 MSB부터 1 → 0 → 1 → 0 순서로 출력된다.

- 예상 타이밍 (A, 1010에 대해)

시각(ns)	관찰 내용
616	A 로드 확인
626	유지 상태 확인, 첫 비트(1) 출력
636	두 번째 비트(0) 출력
646	세 번째 비트(1) 출력
656	네 번째 비트(0) 출력
666	4회 Shift 후 value=0

TB는 상승에지 후 1ns 뒤에 검사한다.

· RTL

```

1 timescale 1ns/1ps
2
3 module piso4(
4     input wire clk, input wire rst, input wire load, input wire enable, input wire [3:0] data_in,
5     output wire serial_out, output reg [3:0] value
6 );
7
8
9
10 assign serial_out = value[3];
11
12 always @(posedge clk) begin
13     if (rst)
14         value <= 4'b0;
15     else if (load)
16         value <= data_in;
17     else if (enable)
18         value <= {value[2:0], 1'b0};
19 end
20
21 endmodule

```

4비트 PISO 레지스터를 구현한 핵심 RTL이다. 리셋, 병렬 로드, 시프트 기능을 수행하며 serial_out은 저장값의 MSB(value[3])에 연결된다. load가 활성화되면 data_in을 저장하고 enable이 활성화되면 저장값을 왼쪽으로 이동시키면서 직렬 데이터를 출력한다.

```

1 `timescale 1ns/1ps
2
3 module input_frontend #(parameter integer STABLE_CYCLES = 20) (
4     input wire clk, input wire rst, input wire button, input wire [7:0] sw,
5     output wire reset, output reg press, output wire [7:0] switches
6 );
7
8
9
10 (* ASYNC_REG = "TRUE" *) reg [1:0] reset_pipe;
11 (* ASYNC_REG = "TRUE" *) reg button_meta, button_sync;
12 (* ASYNC_REG = "TRUE" *) reg [7:0] sw_meta, sw_sync;
13
14 localparam integer WIDTH =
15     (STABLE_CYCLES < 2) ? 1 : $clog2(STABLE_CYCLES);
16
17 reg [WIDTH-1:0] count;
18 reg accepted;
19
20 assign reset = reset_pipe[1];
21 assign switches = sw_sync;
22

```

```

23     always @(posedge clk or posedge rst) begin
24         if (rst)
25             reset_pipe <= 2'b11;
26         else
27             reset_pipe <= {reset_pipe[0], 1'b0};
28     end
29
30     always @(posedge clk or posedge reset) begin
31         if (reset) begin
32             button_meta <= 0;
33             button_sync <= 0;
34             sw_meta <= 0;
35             sw_sync <= 0;
36         end
37         else begin
38             button_meta <= button;
39             button_sync <= button_meta;
40             sw_meta <= sw;
41             sw_sync <= sw_meta;
42         end
43     end

```

```

44     always @(posedge clk or posedge reset) begin
45         if (reset) begin
46             count <= 0;
47             accepted <= 0;
48             press <= 0;
49         end
50         else begin
51             press <= 0;
52
53             if (button_sync == accepted)
54                 count <= 0;
55             else if (count == STABLE_CYCLES - 1) begin
56                 accepted <= button_sync;
57                 count <= 0;
58                 press <= button_sync;
59             end
60             else
61                 count <= count + 1'b1;
62         end
63     end
64 endmodule

```

보드의 스위치와 버튼 입력을 회로 내부 제어 신호로 변환하는 모듈이다. 사용자가 SW1~4로 입력 데이터를 설정하고 SW8로 동작 모드를 선택하면 버튼 입력을 통해 load 또는 enable 신호를 생성하여 PISO 회로를 제어한다.

```

1 `timescale 1ns/1ps
2
3 module lab2_piso #(parameter integer STABLE_CYCLES = 20) (
4     input wire clk, input wire rst, input wire button, input wire [7:0] sw,
5     output wire [7:0] led
6 );
7
8
9
10 wire reset, press;
11 wire [7:0] switches;
12
13 input_frontend #(
14     .STABLE_CYCLES(STABLE_CYCLES)
15 ) inputs (
16     clk,
17     rst,
18     button,
19     sw,
20     reset,
21     press,
22     switches

```

```

23 );
24
25 wire serial_out;
26 wire [3:0] value;
27
28 piso4 core(
29     clk,
30     reset,
31     press && switches[0],
32     press && !switches[0],
33     switches[7:4],
34     serial_out,
35     value
36 );
37
38 assign led = {value, 3'b000, serial_out};
39
40 endmodule

```

프로젝트의 최상위 모듈로서 input_frontend와 piso4를 연결한다. 보드의 스위치 및 버튼 입력을 받아 PISO 회로를 동작시키고 결과를 LED에 표시한다. LED[7:4]는 내부 저장값(value)을, LED[i]은 직렬 출력(serial_out)을 나타낸다.

· TB

```

1 `timescale 1ns/1ps
2
3 module tb_piso4;
4
5     reg clk=0, rst=1;
6     integer checks=0;
7
8     always #5 clk=~clk;
9
10    task step;
11    begin
12        @(posedge clk);
13        #1;
14    end
15    endtask
16
17    task check(input condition, input [8*100-1:0] description);
18    begin
19        checks=checks+1;
20        if (condition != 1'b1) begin
21            $display("LAB2_FAIL %0s time=%0t",description,$time);
22            $fatal(1,"check failed");

```

```

23     end
24 end
25 endtask
26
27 task finish;
28 begin
29     $display("LAB2_PASS piso4 checks=%0d",checks);
30     $finish;
31 end
32 endtask
33
34 initial begin
35     $dumpfile("wave.vcd");
36     $dumpvars(0,tb_piso4);
37 end
38
39 initial begin
40     #100000;
41     $fatal(1,"watchdog timeout");
42 end

```

```

44 reg load=0, enable=0;
45 reg [3:0] data_in=0;
46 wire serial_out;
47 wire [3:0] value;
48
49 piso4 dut(
50     clk,
51     rst,
52     load,
53     enable,
54     data_in,
55     serial_out,
56     value
57 );
58
59 integer word, bit_number;
60
61 initial begin
62     step;
63     check(condition: value===0,description: "reset");
64     rst=0;

```

```

66     for(word=0; word<16; word=word+1) begin
67         data_in=word;
68         load=1;
69         enable=1;
70         step;
71         check(condition: value===word,description: "load beats shift");
72
73         load=0;
74         enable=0;
75         step;
76         check(condition: value===word,description: "hold");
77
78         enable=1;
79
80         for(bit_number=3; bit_number>=0; bit_number=bit_number-1) begin
81             check(condition: serial_out===((word>>bit_number)&1),
82                 description: "MSB first before edge");
83             step;
84         end
85     end

```

```

86     check(condition: value===0,description: "four shifts zero fill");
87 end
88
89 rst=1;
90 load=1;
91 data_in=15;
92 step;
93 check(condition: value===0,description: "reset beats load");
94
95 finish;
96 end
97
98 endmodule

```

- 모든 입력(0~15)을 자동 검사
- Load 기능 검사
- Hold 기능 검사
- Serial 출력 검사
- Shift 결과 검사
- Zero-fill 검사
- 총 114번의 검사를 수행하며 PASS 여부를 자동 판단한다.

· XDC

```

constraints > lab2_piso.xdc
1  set_property PACKAGE_PIN B6 [get_ports {clk}]
2  set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
3
4  set_property PACKAGE_PIN K4 [get_ports {rst}]
5  set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
6
7  set_property PACKAGE_PIN N8 [get_ports {button}]
8  set_property IOSTANDARD LVCMOS33 [get_ports {button}]
9
10 set_property PACKAGE_PIN U4 [get_ports {sw[0]}]
11 set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
12
13 set_property PACKAGE_PIN V4 [get_ports {sw[1]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
15
16 set_property PACKAGE_PIN W1 [get_ports {sw[2]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
18
19 set_property PACKAGE_PIN W4 [get_ports {sw[3]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
21
22 set_property PACKAGE_PIN T1 [get_ports {sw[4]}]

```

```

25 set_property PACKAGE_PIN U2 [get_ports {sw[5]}]
26 set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
27
28 set_property PACKAGE_PIN W3 [get_ports {sw[6]}]
29 set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
30
31 set_property PACKAGE_PIN Y1 [get_ports {sw[7]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
33
34 set_property PACKAGE_PIN N5 [get_ports {led[0]}]
35 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
36
37 set_property PACKAGE_PIN M1 [get_ports {led[1]}]
38 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
39
40 set_property PACKAGE_PIN M3 [get_ports {led[2]}]
41 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
42
43 set_property PACKAGE_PIN M7 [get_ports {led[3]}]
44 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
45

```

```

46 set_property PACKAGE_PIN N7 [get_ports {led[4]}]
47 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
48
49 set_property PACKAGE_PIN M2 [get_ports {led[5]}]
50 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
51
52 set_property PACKAGE_PIN M4 [get_ports {led[6]}]
53 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
54
55 set_property PACKAGE_PIN L4 [get_ports {led[7]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
57
58 create_clock -name trainer_1khz -period 1000000.000 [get_ports clk]
59
60 set_false_path -from [get_ports {rst button sw[*]}]

```

FPGA 핀 번호와 논리 신호를 연결한다.

ex)

- clk → B6
- rst → K4
- button → N8
- sw[7:0] → DIP 스위치
- led[7:0] → LED

또한, 1kHz 클록 제약(create_clock)을 설정한다.

· json

```

1 {
2   "sources": ["src/piso4.v", "src/input_frontend.v", "src/lab2_piso.v"],
3   "testbench": "sim/tb_piso4.v",
4   "simulation_top": "tb_piso4"
5 }
6

```

- 컴파일할 RTL 목록 지정
- 사용할 TB 지정
- 최상위 모듈 지정

· 정상, 변경, 원복 비교

- 정상

‘assign serial_out = value[3];’

```

RUN C:\verilog\bin\verilog.exe -g002 -xll -I C:\Users\kjh36\Documents\Verilog\lab2_05_piso4rc -I C:\Users\kjh36\Documents\Verilog\lab2_05_piso4sim -s
tb_piso4 -o C:\Users\kjh36\Documents\Verilog\lab2_05_piso4\build\simrun-800380404ef43f6d8a38c3dc3d7e\sim.vvp C:\Users\kjh36\Documents\Verilog\lab2_05
_piso4rc\piso4.v C:\Users\kjh36\Documents\Verilog\lab2_05_piso4rc\input_frontend.v C:\Users\kjh36\Documents\Verilog\lab2_05_piso4rc\lab2_piso.v C:\User
s\kjh36\Documents\Verilog\lab2_05_piso4\sim\tb_piso4.v

RUN C:\verilog\bin\vvp.exe -C C:\Users\kjh36\Documents\Verilog\lab2_05_piso4\build\simrun-800380404ef43f6d8a38c3dc3d7e\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_P805 piso4 checks=114
C:\Users\kjh36\Documents\Verilog\lab2_05_piso4\sim\tb_piso4.v:38: $finish called at 970000 (tps)

시뮬레이션 실행 완료. 간략표가이전과 동일하므로 종료의 정답 여부를 확인하십시오.

```

- 변경

‘assign serial_out = value[0];’

MSB 대신 LSB를 출력하도록 수정한다.

```

RUN C:\verilog\bin\verilog.exe -g002 -xll -I C:\Users\kjh36\Documents\Verilog\lab2_05_piso4rc -I C:\Users\kjh36\Documents\Verilog\lab2_05_piso4sim -s
tb_piso4 -o C:\Users\kjh36\Documents\Verilog\lab2_05_piso4\build\simrun-800032404ef43f6d8a38c3dc3d7e\sim.vvp C:\Users\kjh36\Documents\Verilog\lab2_05
_piso4rc\piso4.v C:\Users\kjh36\Documents\Verilog\lab2_05_piso4rc\input_frontend.v C:\Users\kjh36\Documents\Verilog\lab2_05_piso4rc\lab2_piso.v C:\User
s\kjh36\Documents\Verilog\lab2_05_piso4\sim\tb_piso4.v

RUN C:\verilog\bin\vvp.exe -C C:\Users\kjh36\Documents\Verilog\lab2_05_piso4\build\simrun-800032404ef43f6d8a38c3dc3d7e\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_P805 piso4 checks=114
FATAL: C:\Users\kjh36\Documents\Verilog\lab2_05_piso4\sim\tb_piso4.v:22: check failed
Time: 80000 Scope: tb_piso4.check

ERROR: 종료 코드 1: C:\Users\kjh36\Documents\Verilog\lab2_05_piso4\build\simrun-800032404ef43f6d8a38c3dc3d7e\simulation.log

시뮬레이션 실행 완료. 간략표가이전과 동일하므로 종료의 정답 여부를 확인하십시오.
VIVE C:\Users\kjh36\Documents\Verilog\lab2_05_piso4\build\simwave.vcd
시뮬레이션 실행에서 다시 사용됩니다. 종료하면 아무거나 누르세요.

```

- 원복

```

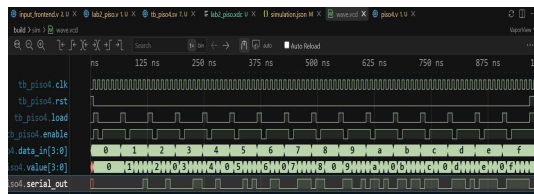
RUN C:\verilog\bin\verilog.exe -g002 -xll -I C:\Users\kjh36\Documents\Verilog\lab2_05_piso4rc -I C:\Users\kjh36\Documents\Verilog\lab2_05_piso4sim -s
tb_piso4 -o C:\Users\kjh36\Documents\Verilog\lab2_05_piso4\build\simrun-ad7cc2af1a04802f5950b0c3d7e\sim.vvp C:\Users\kjh36\Documents\Verilog\lab2_05
_piso4rc\piso4.v C:\Users\kjh36\Documents\Verilog\lab2_05_piso4rc\input_frontend.v C:\Users\kjh36\Documents\Verilog\lab2_05_piso4rc\lab2_piso.v C:\User
s\kjh36\Documents\Verilog\lab2_05_piso4\sim\tb_piso4.v

RUN C:\verilog\bin\vvp.exe -C C:\Users\kjh36\Documents\Verilog\lab2_05_piso4\build\simrun-ad7cc2af1a04802f5950b0c3d7e\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_P805 piso4 checks=114
C:\Users\kjh36\Documents\Verilog\lab2_05_piso4\sim\tb_piso4.v:38: $finish called at 970000 (tps)

시뮬레이션 실행 완료. 간략표가이전과 동일하므로 종료의 정답 여부를 확인하십시오.
VIVE C:\Users\kjh36\Documents\Verilog\lab2_05_piso4\build\simwave.vcd
시뮬레이션 실행에서 다시 사용됩니다. 종료하면 아무거나 누르세요.

```

· 파형 분석



- 확대 구간 : 약 600ns ~ 670ns

시각	이전 상태	입력	다음 상태	serial_out
615ns 예지 직전	0000	1010	1010	1
625ns 예지 직전	1010	유지	1010	1
635ns 예지 직전	1010	shift	0100	0
645ns 예지 직전	0100	shift	1000	1
655ns 예지 직전	1000	shift	0000	0
665ns 예지 직전	0000	shift	0000	0

입력 A가 1010일 때, 615ns에서 A(1010)가 value에 저장된다. 625ns에서는 enable=0이므로 상태가 유지된다. 이후 enable=1 상태에서 Shift가 시작된다. 값은 1010 → 0100 → 1000 → 0000 → 0000 순으로 이동한다. serial_out은 각각 1 → 0 → 1 → 0 순으로 출력된다. 4번 Shift가 끝나면 값은 모두 0이 된다.

· 장비 클록, 핀, LED 동작 예상

- FPGA 핀 정보

신호	핀
clk	B6
rst	K4
button	N8

- 스위치 기능

스위치	역할
SW1~SW4	병렬 입력 (sw[7:4])
SW8	load 선택

SW8=1 → Load

SW8=0 → Shift

- LED 표시

LED	의미
LED[7:4]	현재 value
LED[0]	serial_out

- 예상 보드 동작

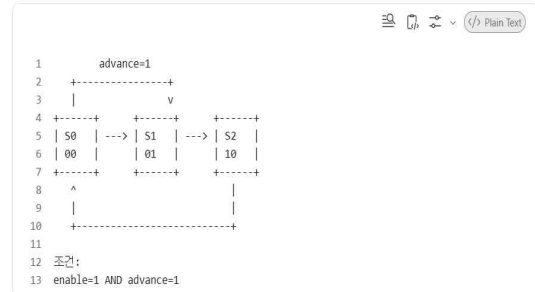
동작	LED[7:4]	LED0
Load	1010	1
Shift 1회	0100	0
Shift 2회	1000	1
Shift 3회	0000	0
Shift 4회	0000	0

6. MOORE

· 회로 목적

Moore State Machine의 동작을 이해하고 구현하는 것을 목표로 한다. Moore 머신은 출력이 현재 입력이 아닌 현재 상태(State)에만 의존하는 순차회로이다. 본 회로는 3개의 상태를 순환한다.

· 상태 전이도



enable=0 또는 advance=0이면 현재 상태를 유지한다. rst=1이면 즉시 S0(00)으로 초기화된다.

· 상태 전이표

현재 상태	enable^advance=1	그 외
00(S0)	01(S1)	00
01(S1)	10(S2)	01
10(S2)	00(S0)	10

· 예상 타이밍 표

검사 시각	입력 조건	예상 상태(value)
6ns	reset	00
7ns	advance=1	00 유지
16ns	enable=1, advance=1	01
26ns	advance=0	01 유지
36ns	enable=0	01 유지
46ns	enable=1, advance=1	10
56ns	enable=1, advance=1	00

· RTL

```

1  `timescale 1ns/1ps
2  module moore_cycle (
3      input wire clk, input wire rst, input wire enable, input wire advance,
4      output reg [1:0] value
5  );
6
7      always @(posedge clk) begin
8          if (rst)
9              value <= 2'b00;
10         else if (enable && advance) begin
11             case (value)
12                 2'b00: value <= 2'b01;
13                 2'b01: value <= 2'b10;
14                 default: value <= 2'b00;
15             endcase
16         end
17     end
18
19 endmodule
  
```

- 상태 레지스터(value) 저장

- rst 처리

- 상태 전이 수행

- S0 → S1 → S2 → S0 순환 구현
- 동기식 always 블록에서 동작하며 상승 에지에서만 상태가 변경된다.

```

1  `timescale 1ns/1ps
2
3  module input_frontend #(parameter integer STABLE_CYCLES = 20) (
4
5      input wire clk,
6      input wire rst,
7      input wire button,
8      input wire [7:0] sw,
9      output wire reset,
10     output reg press,
11     output wire [7:0] switches
12
13 );
14
15 (* ASYNC_REG = "TRUE" *) reg [1:0] reset_pipe;
16 (* ASYNC_REG = "TRUE" *) reg button_meta, button_sync;
17 (* ASYNC_REG = "TRUE" *) reg [7:0] sw_meta, sw_sync;
18
19 localparam integer WIDTH =

```

```

20     (STABLE_CYCLES < 2) ? 1 : $clog2(STABLE_CYCLES);
21
22     reg [WIDTH-1:0] count;
23     reg accepted;
24
25     assign reset = reset_pipe[1];
26     assign switches = sw_sync;
27
28     always @(posedge clk or posedge rst) begin
29         if (rst)
30             reset_pipe <= 2'b11;
31         else
32             reset_pipe <= {reset_pipe[0], 1'b0};
33     end
34
35     always @(posedge clk or posedge reset) begin
36         if (reset) begin
37             button meta <= 0;

```

```

38             button_sync <= 0;
39             sw_meta <= 0;
40             sw_sync <= 0;
41         end
42         else begin
43             button_meta <= button;
44             button_sync <= button_meta;
45             sw_meta <= sw;
46             sw_sync <= sw_meta;
47         end
48     end
49
50     always @(posedge clk or posedge reset) begin
51         if (reset) begin
52             count <= 0;
53             accepted <= 0;

```

```

53         accepted <= 0;
54         press <= 0;
55     end
56     else begin
57         press <= 0;
58
59         if (button_sync == accepted)
60             count <= 0;
61         else if (count == STABLE_CYCLES - 1) begin
62             accepted <= button_sync;
63             count <= 0;
64             press <= button_sync;
65         end
66         else
67             count <= count + 1'b1;
68     end

```

```

69 end
70
71 endmodule

```

- 버튼 디바운싱
 - 스위치 동기화
 - reset 신호 생성
 - enable pulse 생성
- 기계식 버튼의 채터링으로 인한 오동작을 방지한다.

```

1  `timescale 1ns/1ps
2
3  module lab2_moore #(parameter integer STABLE_CYCLES = 20) (
4
5      input wire clk, input wire rst, input wire button, input wire [7:0] sw,
6      output wire [7:0] led
7
8  );
9
10     wire reset, press;
11     wire [7:0] switches;
12
13     input_frontend #(
14         .STABLE_CYCLES(STABLE_CYCLES)
15     ) inputs (
16         clk,
17         rst,
18         button,
19         sw,

```

```

20         reset,
21         press,
22         switches
23     );
24
25     wire [1:0] value;
26
27     moore_cycle core(
28         clk,
29         reset,
30         press,
31         switches[7],
32         value
33     );
34     assign led = {6'b000000, value};
35 endmodule

```

- 입력 모듈 연결
- Moore FSM 연결
- LED 출력 연결

하위 모듈들을 통합하는 역할을 담당한다.

· TB

```
1 timescale 1ns/1ps
2
3 module tb_moore_cycle;
4
5     reg clk=0, rst=1;
6     integer checks=0;
7
8     always #5 clk=~clk;
9
10    task step;
11    begin
12        @(posedge clk);
13        #1;
14    end
15    endtask
16
17    task check(input condition, input [8*100-1:0] description);
18    begin
19        checks=checks+1;
20        if (condition != 1'b1) begin
21            $display("LAB2_FAIL %s time=%0t",description,$time);
22            $fatal(1,"check failed");
23        end
24    end
25    endtask
26
27    task finish;
28    begin
29        $display("LAB2_PASS moore_cycle checks=%0d",checks);
30        $finish;
31    end
32    endtask
33
```

```
34 initial begin
35     $dumpfile("wave.vcd");
36     $dumpvars(0,tb_moore_cycle);
37 end
38
39 initial begin
40     #100000;
41     $fatal(1,"watchdog timeout");
42 end
43
44 reg enable=0, advance=0;
45 wire [1:0] value;
46
47 moore_cycle dut(
48     clk,
49     rst,
50     enable,
51     advance,
52     value
53 );
54
55 integer round;
56
57 initial begin
58     step;
59     check(condition: value===0,description: "reset");
60     rst=0;
61     enable=1;
62
63     advance=1;
64     #1;

```

```
65     check(condition: value===0,description: "input alone cannot change output");
66
67     for(round=0; round<4; round=round+1) begin
68         step;
69         check(condition: value===1,description: "S0 to S1");
70
71         advance=0;
72         step;
73         check(condition: value===1,description: "advance zero holds");
74
75         advance=1;
76         enable=0;
77         step;
78         check(condition: value===1,description: "disabled holds");
79
80         enable=1;
81         step;
82         check(condition: value===2,description: "S1 to S2");
83
84         step;
85         check(condition: value===0,description: "S2 to S0");
86     end
87
88     step;
89     rst=1;
90     step;
91     check(condition: value===0,description: "reset from nonzero");
92
93     finish;

```

```
94     end
95
96 endmodule
97
```

- 입력 자극 생성
- 상태 변화 확인
- 예상값과 실제값 비교
- PASS/FAIL 자동 검증

· json

```
{ simulation.json > ...
1 {
2     "sources": [
3         "src/moore_cycle.v",
4         "src/input_frontend.v",
5         "src/lab2_moore.v"
6     ],
7     "testbench": "sim/tb_moore_cycle.sv",
8     "simulation_top": "tb_moore_cycle"
9 }
```

- 컴파일할 RTL 파일 등록
- 사용할 Test Bench 등록
- simulation_top 지정

· 정상, 변경, 복구 비교표 및 실행 로그

- 비교표

구분	변경 내용	예상 결과
정상	원본 RTL	PASS 23회
변경	S1→S2를 S1→S0로 변경	FAIL 발생
복구	RTL 원상복구	PASS 23회

- 정상 실행 로그

```
Run C:\Verilog\bin\iverilog.EXE -g2012 -hall -I C:\Users\Kjh36\Documents\Verilog\lab2_06\more\src -I C:\Users\Kjh36\Documents\Verilog\lab2_06\more\src -s tb_more_cycle -o C:\Users\Kjh36\Documents\Verilog\lab2_06\more\build\sim\run-aff243fe8b8441856b00d2ef5566a\sim.vvp C:\Users\Kjh36\Documents\Verilog\lab2_06\more\src\more_cycle.v C:\Users\Kjh36\Documents\Verilog\lab2_06\more\src\input_frontend.v C:\Users\Kjh36\Documents\Verilog\lab2_06\more\src\lab2_more.v C:\Users\Kjh36\Documents\Verilog\lab2_06\more\sim\tb_more_cycle.sv

Run C:\Verilog\bin\vpv.EXE -H C:\Users\Kjh36\Documents\Verilog\lab2_06\more\build\sim\run-aff243fe8b8441856b00d2ef5566a\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2 PASS more cycle checks-23
C:\Users\Kjh36\Documents\Verilog\lab2_06\more\sim\tb_more_cycle.sv:38: $finish called at 226000 (tps)

시뮬레이션 실행 완료. 잔여로그가 남아 파형으로 검토의 경우 여부를 확인하세요.
[이미널이 작업에서 다시 사용됩니다, 앞으로는 아무 키나 누르세요.]
```

- 변경 실행

‘2'b01 : value <= 2'b10;’를 ‘2'b01 : value <= 2'b00;’로 변경한다.

```
Run C:\Verilog\bin\iverilog.EXE -g2012 -hall -I C:\Users\Kjh36\Documents\Verilog\lab2_06\more\src -I C:\Users\Kjh36\Documents\Verilog\lab2_06\more\src -s tb_more_cycle -o C:\Users\Kjh36\Documents\Verilog\lab2_06\more\build\sim\run-aff243fe8b8441856b00d2ef5566a\sim.vvp C:\Users\Kjh36\Documents\Verilog\lab2_06\more\src\more_cycle.v C:\Users\Kjh36\Documents\Verilog\lab2_06\more\src\input_frontend.v C:\Users\Kjh36\Documents\Verilog\lab2_06\more\src\lab2_more.v C:\Users\Kjh36\Documents\Verilog\lab2_06\more\sim\tb_more_cycle.sv

Run C:\Verilog\bin\vpv.EXE -H C:\Users\Kjh36\Documents\Verilog\lab2_06\more\build\sim\run-aff243fe8b8441856b00d2ef5566a\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2 FAIL S1 to S2 time=46800
FATAL: C:\Users\Kjh36\Documents\Verilog\lab2_06\more\sim\tb_more_cycle.sv:22: check failed
Time: 46800 Scope: tb_more_cycle.check

[ERROR] 종료 코드 1: C:\Users\Kjh36\Documents\Verilog\lab2_06\more\build\sim\run-aff243fe8b8441856b00d2ef5566a\simulation.log

[이미널이 프로세스 "C:\Users\Kjh36\AppData\Local\Microsoft\WindowsApps\python.EXE "tools\lab1.py", "simulate" "01(7)" 종료되었습니다(종료 코드는 1).]
```

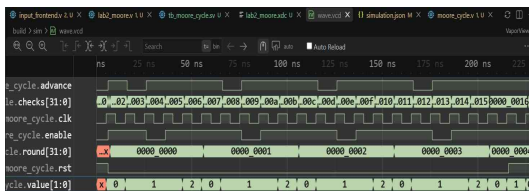
- 복구 결과

```
Run C:\Verilog\bin\iverilog.EXE -g2012 -hall -I C:\Users\Kjh36\Documents\Verilog\lab2_06\more\src -I C:\Users\Kjh36\Documents\Verilog\lab2_06\more\src -s tb_more_cycle -o C:\Users\Kjh36\Documents\Verilog\lab2_06\more\build\sim\run-aff243fe8b8441856b00d2ef5566a\sim.vvp C:\Users\Kjh36\Documents\Verilog\lab2_06\more\src\more_cycle.v C:\Users\Kjh36\Documents\Verilog\lab2_06\more\src\input_frontend.v C:\Users\Kjh36\Documents\Verilog\lab2_06\more\src\lab2_more.v C:\Users\Kjh36\Documents\Verilog\lab2_06\more\sim\tb_more_cycle.sv

Run C:\Verilog\bin\vpv.EXE -H C:\Users\Kjh36\Documents\Verilog\lab2_06\more\build\sim\run-aff243fe8b8441856b00d2ef5566a\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2 PASS more cycle checks-23
C:\Users\Kjh36\Documents\Verilog\lab2_06\more\sim\tb_more_cycle.sv:38: $finish called at 226000 (tps)

시뮬레이션 실행 완료. 잔여로그가 남아 파형으로 검토의 경우 여부를 확인하세요.
[이미널이 작업에서 다시 사용됩니다, 앞으로는 아무 키나 누르세요.]
```

· 파형 해석



- 7ns 구간

이전 상태	입력	다음 상태
00	advance=1	00

advance만 변경되고 클록 상승 에지가 없으므로 상태는 유지된다.

- 16ns 구간

이전 상태	입력	다음 상태
00	enable=1, advance=1	01

유효한 상승 에지가 발생하여 S0 → S1 전이된다.

- 26ns 구간

이전 상태	입력	다음 상태
01	advance=0	01

advance가 비활성 상태이므로 유지된다.

- 36ns 구간

이전 상태	입력	다음 상태
01	enable=0	01

enable이 비활성이라 상태를 유지한다.

- 46ns 구간

이전 상태	입력	다음 상태
01	enable=1, advance=1	10

S1 → S2 전이 발생.

- 56ns 구간

이전 상태	입력	다음 상태
10	enable=1, advance=1	00

S2 → S0 순환 완료.

· 장비 클록, 핀, 예상 LED 동작

- 사용 핀

신호	핀
clk	B6
rst	K4
button(enable)	N8
advance	SW1(sw[7])
출력	LED[1:0]

- 클록 설정

create_clock

period = 1000000 ns

frequency = 1 kHz

보드의 주 클록은 1kHz를 사용한다.

- 예상 LED 동작

조작	LED[1:0]
rst 누름	00
SW1=1, N8 1회	01
SW1=1, N8 2회	10
SW1=1, N8 3회	00
SW1=0, N8 누름	현재 상태 유지

7. MEALY

· 동작 설명

- 상태(state)는 1비트 레지스터이다.
- 입력 bit_in과 현재 state를 이용하여 출력 value를 생성한다.
- enable=1이고 bit_in=1일 때 상승 에지에서 상태가 반전(toggle)된다.
- bit_in=0이면 상태를 유지한다.
- rst=1이면 상태를 0으로 초기화한다.
- 출력 value는 조합회로이므로 클록 없이도 즉시 변화한다.

· 상태 전이표

현재 상태	bit_in	enable	다음 상태
S0(0)	0	0/1	S0
S0(0)	1	0	S0
S0(0)	1	1	S1
S1(1)	0	0/1	S1
S1(1)	1	0	S1
S1(1)	1	1	S0

· 출력표

현재 상태	bit_in	value
S0	0	00
S0	1	10
S1	0	00
S1	1	01

출력은 현재 상태와 입력을 동시에 이용해 계산된다. 따라서 상태가 그대로여도 입력만 바뀌면 출력이 즉시 변한다.

· 예상 타이밍표

시각(ns)	상태(state)	value	(state,value)	설명
6	0	00	000	리셋 완료
7	0	10	010	입력만 1로 변경
16	0	10	010	enable=0 상태 유지
26	1	01	101	enable=1, 상태 전이
27	1	00	100	입력만 0으로 변경
36	1	00	100	상태 유지
37	1	01	101	입력만 1로 변경
46	0	10	010	상태 전이
56	1	01	101	상태 전이
66	0	10	010	입력 1 상태에서 리셋
67	0	00	000	입력 제거

· RTL

```

1 `timescale 1ns/1ps
2 module mealy_toggle (
3
4     input wire clk, input wire rst, input wire enable, input wire bit_in,
5     output reg state, output wire [1:0] value
6
7 );
8
9     always @(posedge clk) begin
10         if (rst)
11             state <= 1'b0;
12         else if (enable && bit_in)
13             state <= ~state;
14         end
15
16         assign value = !bit_in ? 2'b00 :
17             (state ? 2'b01 : 2'b10);
18
19     endmodule
  
```

‘always @(posedge clk)’에서 상태를 저장

하며, rst → state=0, enable=1, bit_in=1
→ state 반전, 그 외 → state 유지를 수행한다. 또한, lassign value 구문으로 Mealy 출력(value)을 생성한다.

```
src > input_frontend.v 1 X lab2_mealy.v 1 U tb_mealy_toggle.v 1 U lab2_mealy_adc.v 1 U
1 `timescale 1ns/1ps
2
3 module input_frontend #(parameter integer STABLE_CYCLES = 20) (
4
5     input wire clk,
6     input wire rst,
7     input wire button,
8     input wire [7:0] sw,
9     output wire reset,
10    output reg press,
11    output wire [7:0] switches
12
13 );
14
15 (* ASYNC_REG = "TRUE" *) reg [1:0] reset_pipe;
16 (* ASYNC_REG = "TRUE" *) reg button_meta, button_sync;
17 (* ASYNC_REG = "TRUE" *) reg [7:0] sw_meta, sw_sync;
18
19 localparam integer WIDTH =
20     (STABLE_CYCLES < 2) ? 1 : $clog2(STABLE_CYCLES);
21
22 reg [WIDTH-1:0] count;
23 reg accepted;
24
25 assign reset = reset_pipe[1];
26 assign switches = sw_sync;
27
28 always @(posedge clk or posedge rst) begin
29     if (rst)
30         reset_pipe <= 2'b11;
31     else
32         reset_pipe <= {reset_pipe[0], 1'b0};
33 end
```

```
34 always @(posedge clk or posedge reset) begin
35     if (reset) begin
36         sw_meta <= 0;
37         sw_sync <= 0;
38     end
39     else begin
40         button_meta <= button;
41         button_sync <= button_meta;
42         sw_meta <= sw;
43         sw_sync <= sw_meta;
44     end
45 end
46
47
48 always @(posedge clk or posedge reset) begin
49     if (reset) begin
50         count <= 0;
51         accepted <= 0;
52         press <= 0;
53     end
54     else begin
55         press <= 0;
56         if (button_sync == accepted)
57             count <= 0;
58         else if (count == STABLE_CYCLES - 1) begin
59             accepted <= button_sync;
60             count <= 0;
61             press <= button_sync;
62         end
63         else
64             count <= count + 1'b1;
65     end
66 end
67 endmodule
```

- reset 동기화
- 버튼 디바운싱
- 스위치 동기화

즉, 기계식 버튼의 잡음(bounce)을 제거하고 FPGA 내부에서 안정적으로 사용할 수 있는 신호를 생성한다.

```
src > input_frontend.v 1 X lab2_mealy.v 1 U tb_mealy_toggle.v 1 U lab2_mealy_adc.v 1 U wave.vvc 1 U
1 `timescale 1ns/1ps
2
3 module lab2_mealy #(parameter integer STABLE_CYCLES = 20) (
4
5     input wire clk, input wire rst, input wire button, input wire [7:0] sw,
6     output wire [7:0] led
7
8 );
9
10 wire reset, press;
11 wire [7:0] switches;
12
13 input_frontend #(
14     .STABLE_CYCLES(STABLE_CYCLES)
15 ) inputs (
16     clk,
17     rst,
18     button,
19     sw,
20     reset,
21     press,
22     switches
23 );
24
25 wire state;
26 wire [1:0] value;
27 mealy_toggle core(
28     clk,
29     reset,
30     press,
31     switches[7],
32     state,
33     value
34 );
35 assign led = {5'b00000, state, value};
36 endmodule
```

- input_frontend 인스턴스
 - mealy_toggle 인스턴스
 - LED 출력 연결
- assign led = {5'b00000, state, value}; 를
통해 LED[2] = state, LED[1:0] = value를
표시한다.

·TB

```
sim > input_frontend.v 1 X lab2_mealy.v 1 U tb_mealy_toggle.v 1 U lab2_mealy_adc.v 1 U wave.vvc 1 U
1 `timescale 1ns/1ps
2
3 module tb_mealy_toggle;
4
5     reg clk=0, rst=1;
6     integer checks=0;
7
8     always #5 clk=~clk;
9
10    task step;
11    begin
12        @(posedge clk);
13        #1;
14    end
15    endtask
16
17    task check(input condition, input [8*100-1:0] description);
18    begin
19        checks=checks+1;
20        if (condition != 1'b1) begin
21            $display("LAB2_FAIL %0s time=%0t", description, $time);
22            $fatal(1, "check failed");
23        end
24    end
25    endtask
26
27    task finish;
28    begin
29        $display("LAB2_PASS mealy_toggle checks=%0d", checks);
30        $finish;
31    end
32    endtask
33
```

```

34     initial begin
35         $dumpfile("wave.vcd");
36         $dumpvars(0,tb_mealy_toggle);
37     end
38
39     initial begin
40         #100000;
41         $fatal(1,"watchdog timeout");
42     end
43
44     reg enable=0, bit_in=0;
45     wire state;
46     wire [1:0] value;
47
48     mealy_toggle dut(
49         clk,
50         rst,
51         enable,
52         bit_in,
53         state,
54         value
55     );
56
57     initial begin
58         step;
59         check(condition: {state,value}===3'b000,description: "reset input zero");
60         rst=0;
61
62         bit_in=1;
63         #1;
64         check(condition: {state,value}===3'b010,description: "S0 input changes bet
65
66         step;
67         check(condition: {state,value}===3'b010,description: "disabled state sti
68
69         enable=1;
70         step;
71         check(condition: {state,value}===3'b101,description: "transition to S1");
72
73         bit_in=0;
74         #1;
75         check(condition: {state,value}===3'b100,description: "S1 input zero imme
76
77         step;
78         check(condition: state==1,description: "input zero holds S1");
79
80         bit_in=1;
81         #1;
82         check(condition: value==1,description: "S1 pre-edge value");
83
84         step;
85         check(condition: {state,value}===3'b010,description: "transition to S0");
86
87         step;
88         check(condition: state==1,description: "toggle again");
89
90         rst=1;
91         step;
92         check(condition: {state,value}===3'b010,description: "reset state with i
93
94         bit_in=0;
95         #1;
96         check(condition: value==0,description: "clear combinational output");
97
98         check(condition: value==0,description: "clear combinational output");
99
100        finish;
101    end
102 endmodule

```

- 클럭 생성
- 입력 자극 제공
- 예상값 자동 비교
- PASS/FAIL 출력

XDC

```

constraints > lab2_mealy.xdc
1  set_property PACKAGE_PIN B6 [get_ports {clk}]
2  set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
3
4  set_property PACKAGE_PIN K4 [get_ports {rst}]
5  set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
6
7  set_property PACKAGE_PIN N8 [get_ports {button}]
8  set_property IOSTANDARD LVCMOS33 [get_ports {button}]
9
10 set_property PACKAGE_PIN U4 [get_ports {sw[0]}]
11 set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
12
13 set_property PACKAGE_PIN V4 [get_ports {sw[1]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
15
16 set_property PACKAGE_PIN W1 [get_ports {sw[2]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
18
19 set_property PACKAGE_PIN W4 [get_ports {sw[3]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
21
22 set_property PACKAGE_PIN T1 [get_ports {sw[4]}]
23 set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
24
25 set_property PACKAGE_PIN U2 [get_ports {sw[5]}]
26 set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
27
28 set_property PACKAGE_PIN W3 [get_ports {sw[6]}]
29 set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
30
31 set_property PACKAGE_PIN Y1 [get_ports {sw[7]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
33
34 set_property PACKAGE_PIN N5 [get_ports {led[0]}]
35 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
36
37 set_property PACKAGE_PIN M1 [get_ports {led[1]}]
38 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
39
40 set_property PACKAGE_PIN M3 [get_ports {led[2]}]
41 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
42
43 set_property PACKAGE_PIN M7 [get_ports {led[3]}]
44 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
45
46 set_property PACKAGE_PIN N7 [get_ports {led[4]}]
47 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
48
49 set_property PACKAGE_PIN M2 [get_ports {led[5]}]
50 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
51
52 set_property PACKAGE_PIN M4 [get_ports {led[6]}]
53 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
54
55 set_property PACKAGE_PIN L4 [get_ports {led[7]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
57
58 create_clock -name trainer_1khz -period 1000000.000 [get_ports clk]
59
60 set_false_path -from [get_ports {rst button sw[*]}]

```

- clk → B6
- rst → K4
- button → N8
- sw[7:0] → DIP Switch
- led[7:0] → LED 에 연결한다.

또한, create_clock 명령으로 1kHz 클럭 제약을 설정한다.

· json

```
simulation.json > ...
1 {
2   "sources": [
3     "src/mealy_toggle.v",
4     "src/input_frontend.v",
5     "src/lab2_mealy.v"
6   ],
7   "testbench": "sim/tb_mealy_toggle.sv",
8   "simulation_top": "tb_mealy_toggle"
9 }
```

sources, testbenc, simulation_to을 통해 컴파일 대상을 지정한다.

· 정상, 변경, 복구 비교표

- 비교표

구분	RTL 상태	결과
정상	원본 코드	PASS 11개
변경	출력값 교환	FAIL 발생
복구	원본 코드 복원	PASS 11개

- 정상 실행

```

RUN C:\Verilog\bin\iverlog.exe -g2012 -wall -I C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\src -I C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\sim -s tb_mealy_toggle -o C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\build\sim\run-10214c69a5472692b6c1a4a0866e\sim.vvp C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\src\mealy_toggle.v C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\src\input_frontend.v C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\src\lab2_mealy.v C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\sim\tb_mealy_toggle.sv

RUN C:\Verilog\bin\iverlog.exe -N C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\build\sim\run-4d9326a13b0243c3b8f6d2ac9c70c18\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2 PASS mealy_toggle checks-11
C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\sim\tb_mealy_toggle.sv:30: $finish called at 67000 (1ps)

시뮬레이션 실행 완료. 진리표-가기검사-파형으로 회로의 정당 여부를 확인하세요.
  
```

- 변경 실행 로그

S0에서 bit_in=1일 때 출력 1, S1에서 bit_in=1일 때 출력 01을 서로 바꾸어 오류를 발생시키도록 지시한다.

```

-I C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\sim -s tb_mealy_toggle -o C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\build\sim\run-8a3a6d83b5e0efdb564acba33916bf5\sim.vvp C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\src\mealy_toggle.v C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\src\input_frontend.v C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\src\lab2_mealy.v C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\sim\tb_mealy_toggle.sv

VCD info: dumpfile wave.vcd opened for output.
LAB2 FAIL S0 input changes between clocks time=7000
FATAL: C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\sim\tb_mealy_toggle.sv:22: check failed
Time: 7000 Scope: tb_mealy_toggle.check

ERROR: 종로 코드 1: C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\build\sim-8a3a6d83b5e0efdb564acba33916bf5\simulation.log
  
```

- 복구 실행 로그

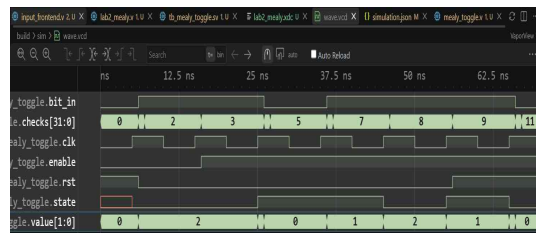
```

-I C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\sim -s tb_mealy_toggle -o C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\build\sim\run-10214c69a5472692b6c1a4a0866e\sim.vvp C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\src\mealy_toggle.v C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\src\input_frontend.v C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\src\lab2_mealy.v C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\sim\tb_mealy_toggle.sv

VCD info: dumpfile wave.vcd opened for output.
LAB2 PASS mealy_toggle checks-11
C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\sim\tb_mealy_toggle.sv:30: $finish called at 67000 (1ps)

시뮬레이션 실행 완료. 진리표-가기검사-파형으로 회로의 정당 여부를 확인하세요.
WAVE C:\Users\kjh36\Documents\Verilog\lab2_07_mealy\build\sim\wave.vcd
타미널이 작업에서 다시 사용됩니다. 알으려면 이후 키나 누르세요.
  
```

· 파형 해석



- 6~16ns 구간 확대

시각	bit_in	state	value
6ns	0	0	00
7ns	1	0	10
16ns	1	0	10

상태는 바뀌지 않았지만, 입력 bit_in이 1이 되면서 조합 출력 value가 즉시 10으로 변경된다.

- 26~37ns 구간 확대

시각	bit_in	state	value
26ns	1	1	01
27ns	0	1	00
37ns	1	1	01

26ns에서 상승 에지로 인해 state가 S1로 전이된다. 이후 27ns에 입력만 0으로 바꾸자 state는 그대로 1이지만 value가 즉시 00으로 변경된다. 37ns에 입력을 다시 1로 올리면 state 변화 없이 value가 01로 변한다.

- 리셋 구간 해석

리셋은 state만 초기화한다. value 계산식

에는 rst가 포함되지 않았으므로 bit_in=1이면 state가 0이어도 value=10이 된다. 따라서 66ns의 {state,value}=010은 정상 결과이다.

· 장비 클록, 핀, 예상 LED 동작

- 클록 및 핀

신호	핀
clk	B6
rst	K4
button	N8
sw[7]	DIP SW1
led[2]	state
led[1:0]	value

- 예상 LED 동작

조작	LED[2:0]
초기화(K4)	000
SW1=1	010
버튼(N8) 1회	101
SW1=0	100
SW1=0 상태 버튼	100
SW1=1 후 버튼	010

LED[2]는 상태(state)를, LED[1:0]은 출력(value)을 나타낸다.

8. SEGMENT SCAN

· 실험 목적

7-Segment Display의 8개 자리를 동시에 구동하지 않고 빠르게 순환 스캔(Multiplexing)하여 여러 숫자를 표시하는 방법을 학습한다. 각 자리 사이에 blank 구간을 삽입하여 고스트 현상을 방지하고, index 상태를 이용해 활성 자리를 순차적으로 변경하는 구조를 이해한다.

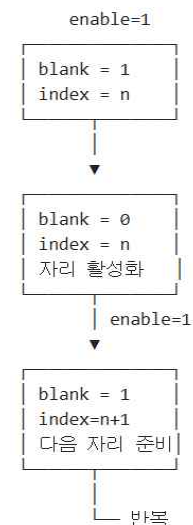
· 상태 변수

상태 변수	의미
index	현재 활성화할 자리 번호(0~7)
blank	자리 변경 전 소등 구간 여부
select	활성 자리 선택 신호
segments	표시할 숫자의 세그먼트 패턴

· 상태 전이표

현재 blank	enable	다음 blank	다음 index	동작
1	1	0	현재값 유지	현재 자리 표시
0	1	1	index+1	다음 자리 준비
X	0	유지	유지	상태 유지
rst=1	X	1	0	초기화

· 상태 전이도



· 예상 타이밍

시간(ns)	index	select	의미
6	0	00	리셋 상태(blank)
16	0	01	숫자 0 표시
26	0	01	enable=0, 유지
36	1	00	blank 구간
46	1	00	enable=0, 유지
56	1	02	숫자 1 표시

첫 두 자리의 동작은 위와 같다.

· 주파수 계산

- 한 자리 ON : 1ms

- Blank : 1ms

- 자리당 주기 : 2ms

- 8자리 전체 순회 : 16ms

따라서 1주파수 = $1 / 0.016$, $2 = 62.5$ Hz
 이므로 각 자리는 약 62.5Hz로 반복 표시 된다.

· RTL

```

1 timescale 1ns/1ps
2
3 // Logical outputs are active-high. Apply board polarity in the wrapper.
4 // Each slot contains one active interval followed by one blank interval.
5 module segment_scan8 (
6
7     input wire clk,
8     input wire rst,
9     input wire enable,
10    input wire [31:0] digits,
11    output reg [7:0] select,
12    output reg [7:0] segments,
13    output reg [2:0] index
14
15);
16
17    reg blank;
18    reg [3:0] nibble;
19
20    always @(posedge clk) begin
21        if (rst) begin
22            index <= 0;
23            blank <= 1'b1;
24        end
25        else if (enable) begin
26            blank <= ~blank;
27            if (!blank)
28                index <= index + 1'b1;
29        end
30    end
31
32    always @* begin
33        nibble = digits >> (index * 4);
    end
    
```

```

34    select = blank ? 8'h00 : (8'h01 << index);
35
36    // abcdefg, dp; decimal point remains off.
37    case (nibble)
38        4'h0: segments = 8'b11111100;
39        4'h1: segments = 8'b01100000;
40        4'h2: segments = 8'b11011010;
41        4'h3: segments = 8'b11110010;
42        4'h4: segments = 8'b01100110;
43        4'h5: segments = 8'b10110110;
44        4'h6: segments = 8'b10111110;
45        4'h7: segments = 8'b11100000;
46        4'h8: segments = 8'b11111110;
47        4'h9: segments = 8'b11110110;
48        4'ha: segments = 8'b11101110;
49        4'hb: segments = 8'b00111110;
50        4'hc: segments = 8'b10011110;
51        4'hd: segments = 8'b01111010;
52        4'he: segments = 8'b10011110;
53        4'hf: segments = 8'b10001110;
54        default: segments = 8'h00;
55    endcase
56
57    end
58 endmodule
    
```

- index, blank 생성

- digits에서 현재 숫자 추출

- select 생성

- 7-Segment 패턴 생성

```

1 timescale 1ns/1ps
2
3 module input_frontend #(parameter integer STABLE_CYCLES = 20) (
4
5     input wire clk,
6     input wire rst,
7     input wire button,
8     input wire [7:0] sw,
9     output wire reset,
10    output reg press,
11    output wire [7:0] switches
12
13);
14
15    (* ASYNC_REG = "TRUE" *) reg [1:0] reset_pipe;
16    (* ASYNC_REG = "TRUE" *) reg button_meta, button_sync;
17    (* ASYNC_REG = "TRUE" *) reg [7:0] sw_meta, sw_sync;
18
19    localparam integer WIDTH =
20        (STABLE_CYCLES < 2) ? 1 : $clog2(STABLE_CYCLES);
21
22    reg [WIDTH-1:0] count;
23    reg accepted;
24
25    assign reset = reset_pipe[1];
26    assign switches = sw_sync;
27
28    // Asynchronous assertion; release follows two main-clock edges.
29    always @(posedge clk or posedge rst) begin
30        if (rst)
31            reset_pipe <= 2'b11;
32        else
33            reset_pipe <= {reset_pipe[0], 1'b0};
    end
    
```

```

34    end
35
36    always @(posedge clk or posedge reset) begin
37        if (reset) begin
38            button_meta <= 0;
39            button_sync <= 0;
40            sw_meta <= 0;
41            sw_sync <= 0;
42        end
43        else begin
44            button_meta <= button;
45            button_sync <= button_meta;
46            sw_meta <= sw;
47            sw_sync <= sw_meta;
48        end
49    end
50
51    always @(posedge clk or posedge reset) begin
52        if (reset) begin
53            count <= 0;
54            accepted <= 0;
55            press <= 0;
56        end
57        else begin
58            press <= 0;
59
60            if (button_sync == accepted)
61                count <= 0;
62            else if (count == STABLE_CYCLES - 1) begin
63                accepted <= button_sync;
64                count <= 0;
            end
        end
    end
    
```

```

65        press <= button_sync;
66    end
67    else
68        count <= count + 1'b1;
69    end
70 end
71 endmodule
72
73
    
```

- 버튼 동기화

- 스위치 동기화

- 디바운싱

- reset 생성

TB

```

1 `timescale 1ns/1ps
2
3 module segment_scan8 (
4
5     input wire clk,
6     input wire rst,
7     input wire enable,
8     input wire [31:0] digits,
9     output reg [7:0] select,
10    output reg [7:0] segments,
11    output reg [2:0] index
12
13 );
14
15 reg blank;
16 reg [3:0] nibble;
17
18 always @(posedge clk) begin
19     if (rst) begin
20         index <= 0;
21         blank <= 1'b1;
22     end
23     else if (enable) begin
24         blank <= ~blank;
25         if (!blank)
26             index <= index + 1'b1;
27     end
28 end
29
30 always @* begin
31     nibble = digits >> (index * 4);
32     select = blank ? 8'h00 : (8'h01 << index);
33

```

```

34 // abcdefg, dp; decimal point remains off.
35 case (nibble)
36     4'h0: segments = 8'b11111100;
37     4'h1: segments = 8'b01100000;
38     4'h2: segments = 8'b11011010;
39     4'h3: segments = 8'b11110010;
40     4'h4: segments = 8'b01100110;
41     4'h5: segments = 8'b10110110;
42     4'h6: segments = 8'b10111110;
43     4'h7: segments = 8'b11100000;
44     4'h8: segments = 8'b11111110;
45     4'h9: segments = 8'b11110110;
46     4'ha: segments = 8'b11101110;
47     4'hb: segments = 8'b00111110;
48     4'hc: segments = 8'b10011100;
49     4'hd: segments = 8'b01111010;
50     4'he: segments = 8'b10011110;
51     4'hf: segments = 8'b10001110;
52     default: segments = 8'h00;
53 endcase
54 end
55
56 endmodule

```

- 입력 frontend 연결
- segment_scan8 연결
- LED 출력 연결
- seg_com 극성 변환

```

1 `timescale 1ns/1ps
2
3 module tb_segment_scan8;
4
5     reg clk=0, rst=1;
6     integer checks=0;
7
8     always #5 clk=~clk;
9
10    task step;
11    begin
12        @(posedge clk);
13        #1;
14    end
15    endtask
16
17    task check(input condition, input [8*100-1:0] description);
18    begin
19        checks=checks+1;
20        if (condition != 1'b1) begin
21            $display("LAB2_FAIL %0s time=%0t",description,$time);
22            $fatal(1,"check failed");
23        end
24    end
25    endtask
26
27    task finish;
28    begin
29        $display("LAB2_PASS segment_scan8 checks=%0d",checks);
30        $finish;
31    end
32    endtask
33

```

```

34 initial begin
35     $dumpfile("wave.vcd");
36     $dumpvars(0,tb_segment_scan8);
37 end
38
39 initial begin
40     #100000;
41     $fatal(1,"watchdog timeout");
42 end
43
44 reg enable=0;
45 reg [31:0] digits=32'h76543210;
46 wire [7:0] select, segments;
47 wire [2:0] index;
48
49 segment_scan8 dut(
50     clk,
51     rst,
52     enable,
53     digits,
54     select,
55     segments,
56     index
57 );
58
59 reg [7:0] expected[0:15];
60 integer bank, position, lap;
61
62 initial begin
63     // Truth table values, ordered 0..F with abcdefg and dp.
64     expected[0]=8'hfc; expected[1]=8'h60; expected[2]=8'hda; expected[3]=8'hf2;

```

```

65     expected[4]=8'h66; expected[5]=8'hb6; expected[6]=8'hbe; expected[7]=8'ha0;
66     expected[8]=8'hfe; expected[9]=8'hfe; expected[10]=8'hae; expected[11]=8'h3e;
67     expected[12]=8'h9c; expected[13]=8'h7a; expected[14]=8'h9e; expected[15]=8'h8e;
68
69 step;
70 check(condition: select==0 && index==0,description: "reset blanks digit zero");
71 rst=0;
72
73 for(bank=0;bank<2;bank=bank+1) begin
74     digits=bank ? 32'hfedcba98 : 32'h76543210;
75
76     for(lap=0;lap<2;lap=lap+1) begin
77         for(position=0;position<8;position=position+1) begin
78
79             enable=1;
80             step;
81
82             check(condition: index==position,description: "position order");
83             check(condition: select==1<<(position),description: "exactly intended digit active");
84             check(condition: segments==expected[bank*8+position],description: "hex truth table");
85
86             enable=0;
87             step;
88
89             check(condition: select==1<<(position),description: "enable zero holds active slot");
90
91             enable=1;
92             step;
93

```

```

94         check(condition: select==0,description: "blank interval between every digit");
95     end
96     enable=0;
97     step;
98     check(condition: select==0,description: "blank interval hold");
99 end
100
101 end
102
103
104 enable=1;
105 step;
106 rst=1;
107 step;
108
109 check(condition: select==0 && index==0,description: "reset from active scan");
110
111 finish;
112 end
113
114 endmodule

```

- index 순서
 - select 동작
 - segment 패턴
 - blank 구간
 - enable=0 유지
 - reset 동작
- 위 항목들을 검사한다.

· XDC

```

1  set_property PACKAGE_PIN B6 [get_ports {clk}]
2  set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
3
4  set_property PACKAGE_PIN K4 [get_ports {rst}]
5  set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
6
7  set_property PACKAGE_PIN N8 [get_ports {button}]
8  set_property IOSTANDARD LVCMOS33 [get_ports {button}]
9
10 set_property PACKAGE_PIN U4 [get_ports {sw[0]}]
11 set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
12
13 set_property PACKAGE_PIN V4 [get_ports {sw[1]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
15
16 set_property PACKAGE_PIN W1 [get_ports {sw[2]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
18
19 set_property PACKAGE_PIN W4 [get_ports {sw[3]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
21
22 set_property PACKAGE_PIN T1 [get_ports {sw[4]}]
23 set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
24
25 set_property PACKAGE_PIN U2 [get_ports {sw[5]}]
26 set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
27
28 set_property PACKAGE_PIN W3 [get_ports {sw[6]}]
29 set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
30
31 set_property PACKAGE_PIN Y1 [get_ports {sw[7]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
33

```

```

34 set_property PACKAGE_PIN N5 [get_ports {led[0]}]
35 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
36
37 set_property PACKAGE_PIN M1 [get_ports {led[1]}]
38 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
39
40 set_property PACKAGE_PIN M3 [get_ports {led[2]}]
41 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
42
43 set_property PACKAGE_PIN M7 [get_ports {led[3]}]
44 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
45
46 set_property PACKAGE_PIN N7 [get_ports {led[4]}]
47 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
48
49 set_property PACKAGE_PIN M2 [get_ports {led[5]}]
50 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
51
52 set_property PACKAGE_PIN M4 [get_ports {led[6]}]
53 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
54
55 set_property PACKAGE_PIN L4 [get_ports {led[7]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
57
58 set_property PACKAGE_PIN H2 [get_ports {seg_data[0]}]
59 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[0]}]
60
61 set_property PACKAGE_PIN J7 [get_ports {seg_data[1]}]
62 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[1]}]
63
64 set_property PACKAGE_PIN J3 [get_ports {seg_data[2]}]
65 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[2]}]

```

```

67 set_property PACKAGE_PIN J1 [get_ports {seg_data[3]}]
68 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[3]}]
69
70 set_property PACKAGE_PIN E4 [get_ports {seg_data[4]}]
71 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[4]}]
72
73 set_property PACKAGE_PIN E2 [get_ports {seg_data[5]}]
74 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[5]}]
75
76 set_property PACKAGE_PIN F5 [get_ports {seg_data[6]}]
77 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[6]}]
78
79 set_property PACKAGE_PIN F1 [get_ports {seg_data[7]}]
80 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[7]}]
81
82 set_property PACKAGE_PIN K5 [get_ports {seg_com[0]}]
83 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[0]}]
84
85 set_property PACKAGE_PIN K3 [get_ports {seg_com[1]}]
86 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[1]}]
87
88 set_property PACKAGE_PIN K1 [get_ports {seg_com[2]}]
89 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[2]}]
90
91 set_property PACKAGE_PIN L6 [get_ports {seg_com[3]}]
92 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[3]}]
93
94 set_property PACKAGE_PIN G3 [get_ports {seg_com[4]}]
95 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[4]}]
96
97 set_property PACKAGE_PIN G1 [get_ports {seg_com[5]}]
98 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[5]}]

```

```

99
100 set_property PACKAGE_PIN H6 [get_ports {seg_com[6]}]
101 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[6]}]
102
103 set_property PACKAGE_PIN H4 [get_ports {seg_com[7]}]
104 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[7]}]
105
106 create_clock -name trainer_1khz -period 1000000.000 [get_ports clk]
107
108 set_false_path -from [get_ports {rst button sw[*]}]

```

FPGA 내부 신호를 외부 핀과 연결한다.

· json

```

simulation.json > ...
1  {
2      "sources": [
3          "src/segment_scan8.v",
4          "src/input_frontend.v",
5          "src/lab2_segment_scan.v"
6      ],
7      "testbench": "sim/tb_segment_scan8.sv",
8      "simulation_top": "tb_segment_scan8"
9  }

```

- sources
- 2testbench
- 3simulation_top

Icarus Verilog가 위 파일의 컴파일을 지정한다.

· 정상, 변경, 복구 비교표

구분	결과	검사 수	종료 시각
정상	PASS	194	1306ns
변경	FAIL	초기 검사 실패	즉시 종료
복구	PASS	194	1306ns

- 정상 실행 로그

```

# Run C:\verilog\bin\verilog.DEL -p002 -hall -1 C:\Users\kjh\Documents\VerilogLab_00_segment_scan\src -1 C:\Users\kjh\Documents\VerilogLab_00_seg
ment_scan\sim -1 th_segment_scan -o C:\Users\kjh\Documents\VerilogLab_00_segment_scan\build\sim-run-cff2b0b7baf7a55c113b4bde0\sim.vvp C:\Users\
kjh\Documents\VerilogLab_00_segment_scan\src\segment_scan.v C:\Users\kjh\Documents\VerilogLab_00_segment_scan\src\input_frontend.v C:\Users\kjh\
Documents\VerilogLab_00_segment_scan\src\l02_segment_scan.v C:\Users\kjh\Documents\VerilogLab_00_segment_scan\src\l01_segment_scan.v
Run C:\verilog\bin\verilog.DEL -R C:\Users\kjh\Documents\VerilogLab_00_segment_scan\build\sim-run-cff2b0b7baf7a55c113b4bde0\sim.vvp
V02 info: dumpfile was not opened for output.
LAB2 P005 segment_scan checks=194
C:\Users\kjh\Documents\VerilogLab_00_segment_scan\src\l01_segment_scan.v:30: $finish called at 100000 (ps)

시뮬레이션 실행 완료. 전파표시/가측시간/가측으로 회로의 작동 상태를 확인하십시오.
LAB5 C:\Users\kjh\Documents\VerilogLab_00_segment_scan\build\sim-run-cff2b0b7baf7a55c113b4bde0\sim.vcd
파라미터 작업에서 다시 사용됩니다. 완료되면 아무 키나 누르세요.
```

- 오류 실행 로그

‘blank ? 8'h00 :’ 부분을 제거하여 오류를 발생시킨다.

```

# Run C:\verilog\bin\verilog.DEL -p002 -hall -1 C:\Users\kjh\Documents\VerilogLab_00_segment_scan\src -1 C:\Users\kjh\Documents\VerilogLab_00_seg
ment_scan\sim -1 th_segment_scan -o C:\Users\kjh\Documents\VerilogLab_00_segment_scan\build\sim-run-cff2b0b7baf7a55c113b4bde0\sim.vvp C:\Users\
kjh\Documents\VerilogLab_00_segment_scan\src\segment_scan.v C:\Users\kjh\Documents\VerilogLab_00_segment_scan\src\input_frontend.v C:\Users\kjh\
Documents\VerilogLab_00_segment_scan\src\l02_segment_scan.v C:\Users\kjh\Documents\VerilogLab_00_segment_scan\src\l01_segment_scan.v
C:\Users\kjh\Documents\VerilogLab_00_segment_scan\src\segment_scan.v:30: syntax error
C:\Users\kjh\Documents\VerilogLab_00_segment_scan\src\segment_scan.v:30: error: reformed statement

ERR00: 종료 코드 2: C:\Users\kjh\AppData\Local\Microsoft\WindowsApps\python.DEL "tools\lab1.py", "simulate" "01(가) 종료되었습니다(종료 코드: 1).
파라미터 작업에서 다시 사용됩니다. 완료되면 아무 키나 누르세요.
```

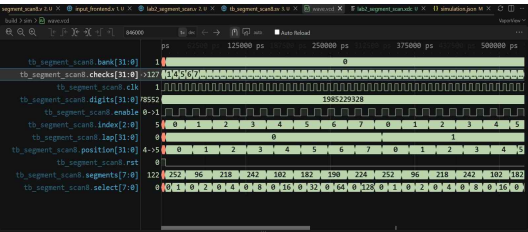
- 복구 후 로그

```

# Run C:\verilog\bin\verilog.DEL -p002 -hall -1 C:\Users\kjh\Documents\VerilogLab_00_segment_scan\src -1 C:\Users\kjh\Documents\VerilogLab_00_seg
ment_scan\sim -1 th_segment_scan -o C:\Users\kjh\Documents\VerilogLab_00_segment_scan\build\sim-run-cff2b0b7baf7a55c113b4bde0\sim.vvp C:\Users\
kjh\Documents\VerilogLab_00_segment_scan\src\segment_scan.v C:\Users\kjh\Documents\VerilogLab_00_segment_scan\src\input_frontend.v C:\Users\kjh\
Documents\VerilogLab_00_segment_scan\src\l02_segment_scan.v C:\Users\kjh\Documents\VerilogLab_00_segment_scan\src\l01_segment_scan.v
Run C:\verilog\bin\verilog.DEL -R C:\Users\kjh\Documents\VerilogLab_00_segment_scan\build\sim-run-cff2b0b7baf7a55c113b4bde0\sim.vvp
V02 info: dumpfile was not opened for output.
LAB2 P005 segment_scan checks=194
C:\Users\kjh\Documents\VerilogLab_00_segment_scan\src\l01_segment_scan.v:30: $finish called at 100000 (ps)

시뮬레이션 실행 완료. 전파표시/가측시간/가측으로 회로의 작동 상태를 확인하십시오.
LAB5 C:\Users\kjh\Documents\VerilogLab_00_segment_scan\build\sim-run-cff2b0b7baf7a55c113b4bde0\sim.vcd
파라미터 작업에서 다시 사용됩니다. 완료되면 아무 키나 누르세요.
```

· 파형 해석



- 리셋 구간

신호	값
rst	1
index	0
blank	1
select	00

모든 자리가 꺼진 초기 상태이다.

- 첫 번째 자리 활성화

신호	값
rst	0
index	0
select	01
segments	FC

숫자 0이 표시된다.

- enable=0 유지 구간

이전 상태	다음 상태
index=0	index=0
select=01	select=01

- Blank 구간

신호	값
index	1
select	00

다음 자리 전환을 준비하는 소등 구간이다.

- 두 번째 자리 활성화

신호	값
index	1
select	02
segments	60

· 장비 클럭, 핀, 예상 LED 동작

- 클럭 입력 및 핀

신호	핀
clk	B6
rst	K4
button	N8

- 스위치 핀

스위치	기능
SW1~SW4	첫 자리 숫자 입력
sw[7:4]	표시 숫자 결정

- LED 동작

‘assign led = {5'b00000, index};’

index	LED[2:0]
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

9. 참고문헌

· 전자전기컴퓨터설계실험 II 교안

· [Git-Hub](#)